

# **dailyfeed backend**

**주요 백엔드 기능 및 설계 관련 설명**

**2025/12/08 (정순구)**

# 핵심 포인트

## 설계시 핵심아이디어

+ 카프카 통신 시 에러 처리 방식

+ 논리적 도메인 구조의 서비스를 스케일링 용도(저장 vs 통계/조회)별 분리

:: 논리적인 구조로만 분해할 경우 예상치 못한 트래픽 급증시 특정 서비스 전체가 스케일 아웃 되는 구조는 지양

:: Read / Write 용도의 레플리케이션 그룹을 분리해서 스케일링되도록 구성 (e.g. Read 부하가 심할때는 timeline 그룹의 스케일 아웃, Write 부하가 심할때는 content 그룹의 스케일 아웃)

:: 저장소까지 분리한 완전한 MSA 는 아니지만, 부하(로드)의 성격별 레플리케이션 그룹을 지정

:: 7인8각 게임과 같은 모놀리딕 구조는 배제 (내가 디지면 너도 디지는거야. 잘해. vs 너는 이거해, 나는 이거할께)

+ 통신레벨과 애플리케이션레벨 결합도 분리 (L4 레벨과 L7 레벨의 분리)

:: 애플리케이션에서는 통신레벨을 알 필요가 없고, 통신레벨에서는 전달받은 값만 받아서 통신이라는 역할만을 수행

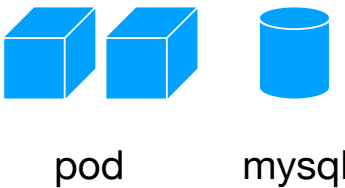
:: 통신레벨에서는 통신만 수행하고, 익셉션은 애플리케이션 계층으로 throw

+ no common → 주요 서브모듈로 모듈화

# Overview

## dailyfeed-member-svc

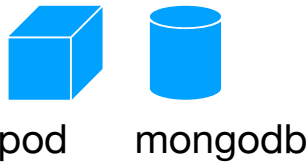
+ 회원가입,로그인,로그아웃



min: 1  
max: 2

## dailyfeed-activity-svc

+ 활동기록 저장/조회/수정



min: 1  
max: 2

season2

- season2 페이지 예정

## dailyfeed-content-svc : 글 생성/수정/삭제 전용 서비스

+ 글 작성/수정/삭제

+ 댓글 작성/수정/삭제

+ 댓글의 답글 작성/수정/삭제



min: 1  
max: 3

## dailyfeed-timeline-svc : 사용자 맞춤 조회/통계 쿼리 전용 서비스

+ 팔로우 중인 멤버들의 최근 작성글(follow feed)

+ 지금 가장 인기있는 글 (most popular)

+ 댓글 많은 글 (most commented)

+ 나의 작성글들 (my feed)

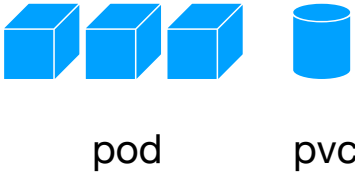


min: 1  
max: 7

## dailyfeed-image-svc : 이미지 조회/업로드/삭제 서비스

+ 프로필/썸네일 이미지 업로드/수정/삭제/조회

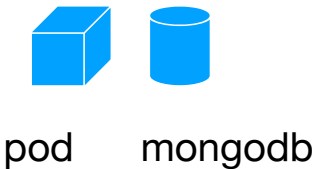
+ timeline 조회, 추천회원 프로필 목록 조회, 나의 프로필에서 이미지 조회



min: 3  
max: 5

## dailyfeed-search-svc : 본문 검색 서비스

+ Full Text Search 를 위한 별도의 서비스



min: 1  
max: 3



프로젝트 github

# 프로젝트 github

프로젝트 설명 github

<https://github.com/alpha3002025/dailyfeed-readme-season1>

프로젝트 설치 github

<https://github.com/alpha3002025/dailyfeed-installer>

dailyfeed-frontend-svc

<https://github.com/alpha3002025/dailyfeed-frontend-svc>

dailyfeed-member-svc

<https://github.com/alpha3002025/dailyfeed-member-svc>

dailyfeed-content-svc

<https://github.com/alpha3002025/dailyfeed-content-svc>

dailyfeed-timeline-svc

<https://github.com/alpha3002025/dailyfeed-timeline-svc>

dailyfeed-search-svc

<https://github.com/alpha3002025/daiyfeed-search-svc>

dailyfeed-image-svc

<https://github.com/alpha3002025/dailyfeed-image-svc>

dailyfeed-activity-svc

<https://github.com/alpha3002025/dailyfeed-activity-svc>

dailyfeed-batch-svc

<https://github.com/alpha3002025/dailyfeed-batch-svc>

## git clone

- 각 서브모듈 재귀적 clone 방법
- git submodule update/pull 방법
- 커밋/업데이트 방법



# 새로운 환경에서 클론(clone)

## 새로운 환경에서 클론 (clone)

# Submodule과 함께 클론 (content-svc, timeline-svc, activity-svc, search-svc, image-svc 도 모두 이와 같이 해주시면 됩니다.)

```
git clone --recurse-submodules http://github.com/alpha3002025/dailyfeed-member-svc
```

# 또는 클론 후 submodule 초기화

```
git clone dailyfeed-member-svc
```

```
cd dailyfeed-member-svc
```

```
git submodule init
```

```
git submodule update
```

# 서브모듈 업데이트

모든 **submodule** 을 최신 상태로 업데이트

```
git submodule update --remote
```

특정 **submodule** 만 업데이트

```
git submodule update --remote dailyfeed-code
```

또는 다음과 같이 **main** 브랜치를 **pull** 하는 코드를 실행해주세요.

```
source git-pull-all.sh
```



# 새로운 원격 submodule repository 를 현재 repository에 submodule 로 추가

e.g. dailyfeed-redis-support

모든 submodule 을 clone 및 gitmodules 등록

```
git submodule add https://github.com/alpha3002025/dailfyeed-redis-support dailyfeed-redis-support
```

**settings.gradle.kts** 에는 다음 구문을 추가합니다.

```
include("dailyfeed-redis-support")
```

**설치방법**

**(dailyfeed-installer)**

# 설치방법 (local k8s)

local PC 내에 kind 클러스터기반의 kubernetes 환경을 구축하는 방식입니다.

## (1) orbstack 또는 docker desktop 설치

orbstack 또는 docker desktop 을 설치합니다.

## (2) dailyfeed-installer 리포지터리 clone

dailyfeed-installer github : <https://github.com/alpha3002025/dailyfeed-installer>

### (3.1) install 스크립트 실행

```
source local-install-infra-and-app.sh main
```

또는 다음과 같이 수행합니다.

```
source local-install-infra-and-app.sh {이미지태그명}
```

### (3.2) 만약 infra 만 설치하려 할 경우

```
cd dailyfeed-infrastructure
```

```
source install-local.sh
```

### (3.3) 애플리케이션만 설치하려 할 경우

```
cd dailyfeed-app-helm
```

```
source install-local.sh {이미지 태그명}
```



# 삭제 (local k8s)

**(1) dailyfeed-infrastructure**

cd dailyfeed-infrastructure

source uninstall-all-infra.sh

# 설치방법 (local was)

local PC 내에 infra(mysql,redis,kafka,mongodb) 를 docker-compose 로 설치한 후 개별 서비스들은 WAS 로 각각 구동할때의 실행방법입니다.

## (1) orystack 또는 docker desktop 설치

orystack 또는 docker desktop 을 설치합니다.

## (2) dailyfeed-installer 리포지터리 clone

dailyfeed-installer github : <https://github.com/alpha3002025/dailyfeed-installer>

## (3) infra (mysql,redis,kafka,mongodb) 설치

```
cd dailyfeed-infrastructure/docker/mysql-mongodb-redis
```

```
docker-compose up -d
```

## (4) 각각의 개별 서비스에서는 다음의 명령을 수행하시면 됩니다.

e.g. dailyfeed-member-svc

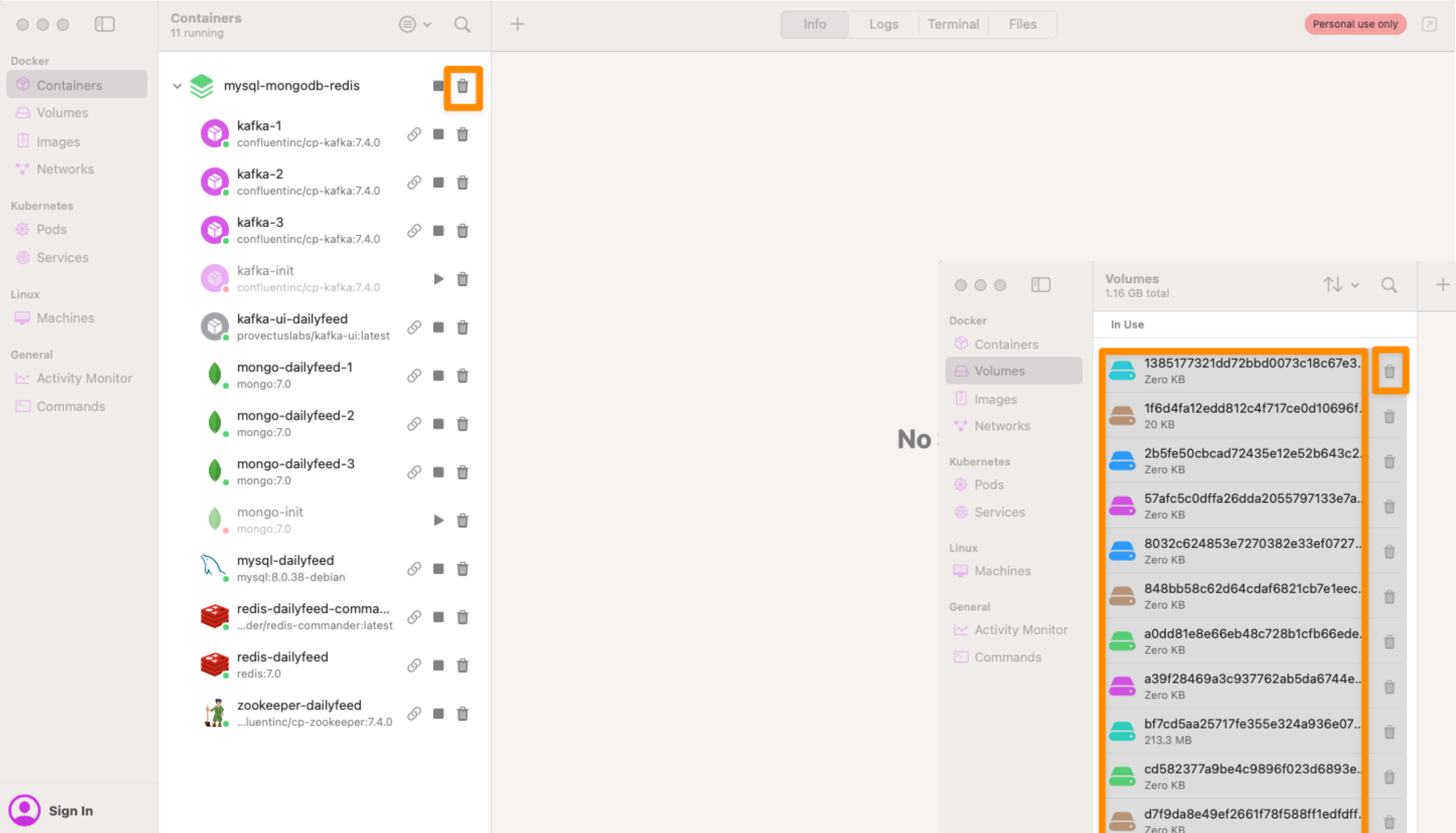
```
cd dailyfeed-member-svc
```

```
source git-pull-all.sh
```

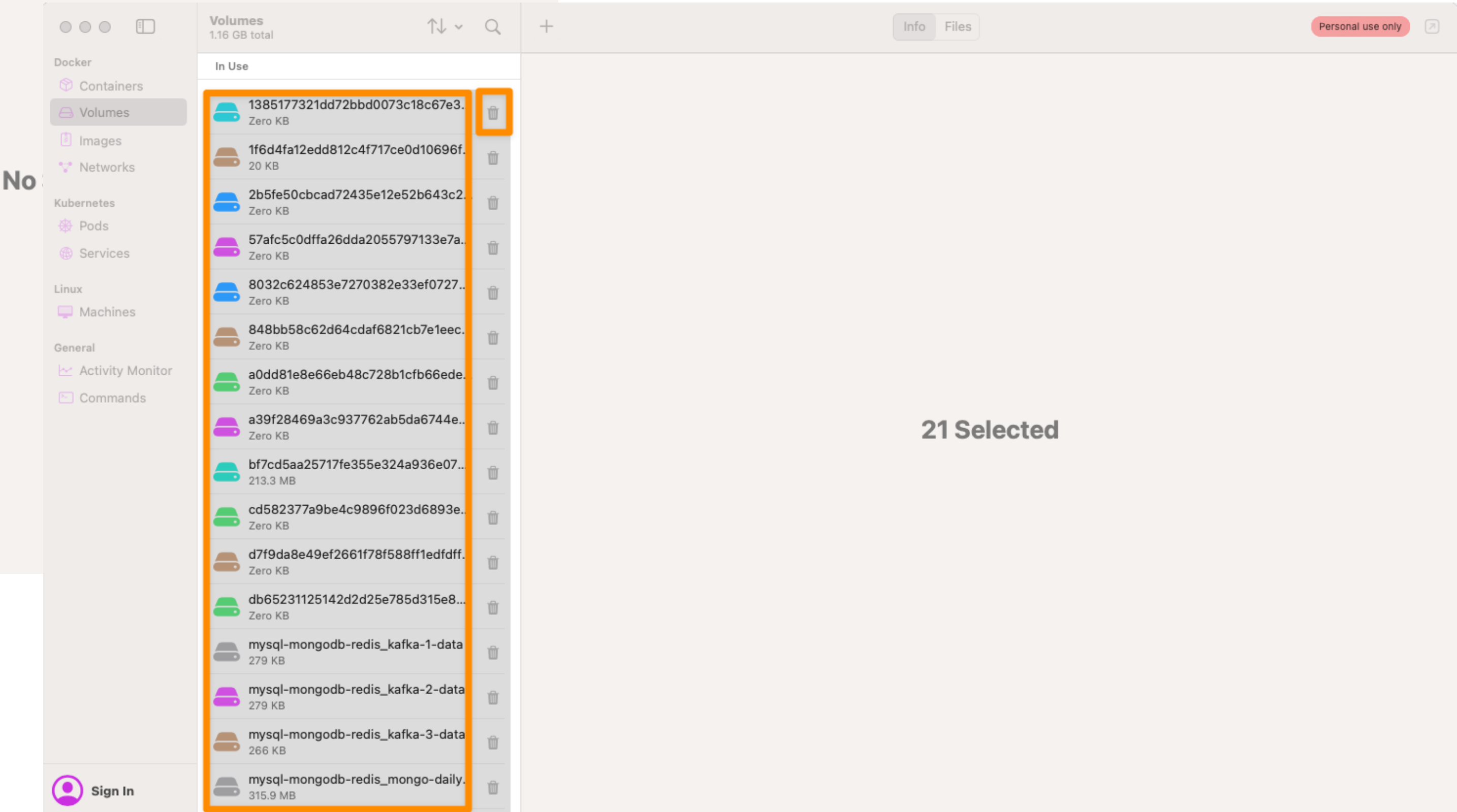
```
source run-local-was.sh
```

# 삭제 (local was)

orbstack 에서 컨테이너, volume 삭제



container 삭제



volume 삭제

cmd + A 를 통해 모든 볼륨 선택 후 휴지통 버튼을 클릭해서 모든 볼륨을 삭제



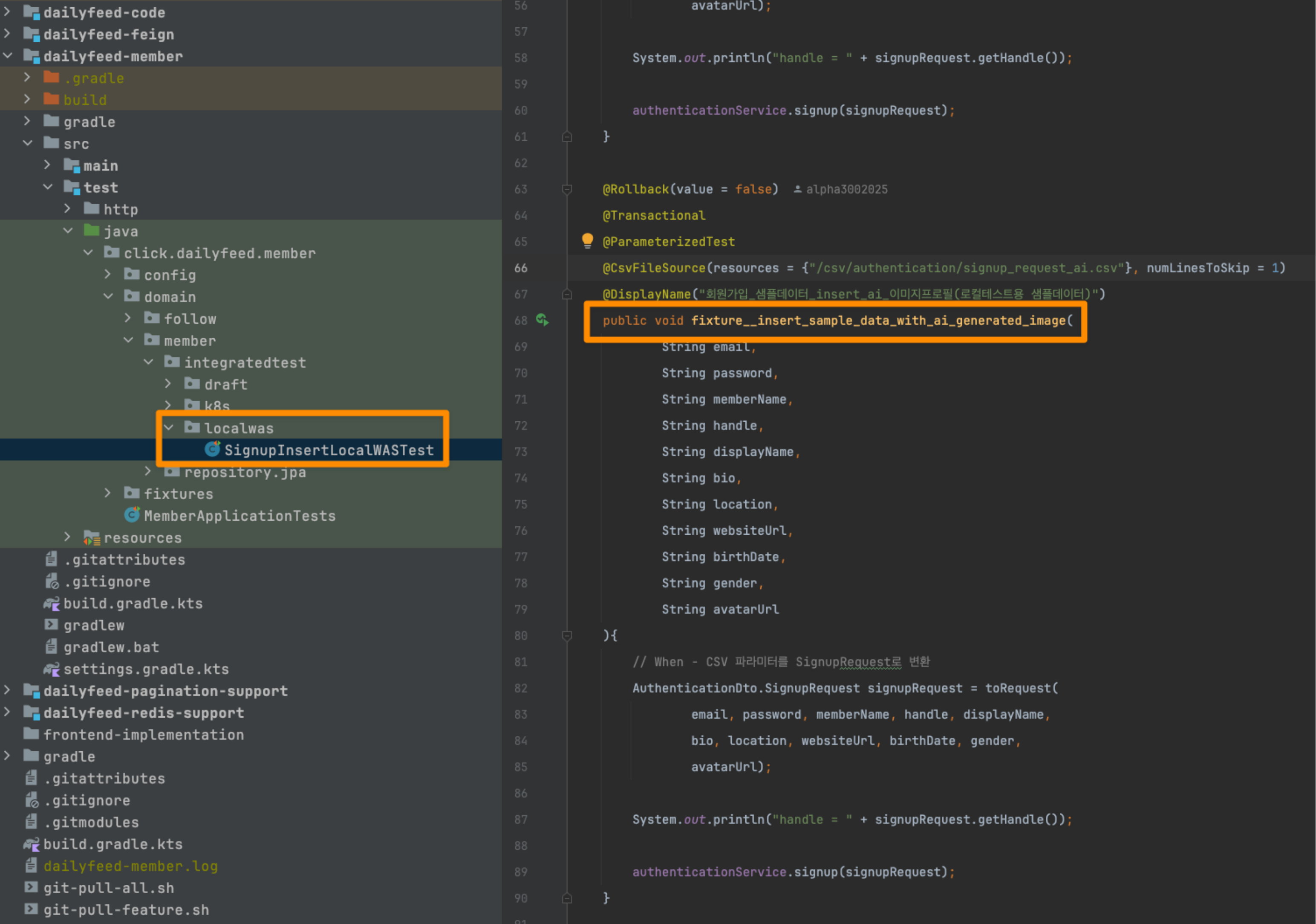
**테스트 회원 데이터 insert**

# 테스트 회원 데이터 insert (local-was)

회원을 50 명 정도를 직접 사람이 일일이 가입해서 follow/unfollow 하는 것을 테스트하기는 쉽지 않습니다.

이런 이유로 Test 코드로 테스트 데이터를 insert 하는 코드를 만들어두었습니다.

(1) local-was 프로필 (로컬에서 docker-compose 로 인프라 설치한 곳에서 개별 was 들을 구동시)



src/test/java/click.dailyfeed.member/domain/  
member/integratedtest/localwas/  
SignupInsertLocalWASTest.java 를 선택합니다.

테스트 클래스 내에 다음 테스트 메서드를 실행합니다.

fixture\_\_insert\_sample\_data\_with\_ai\_generated\_image()

이렇게 하면 docker-compose 로 설치한 infra (mysql) 에  
회원 데이터들이 insert 됩니다.

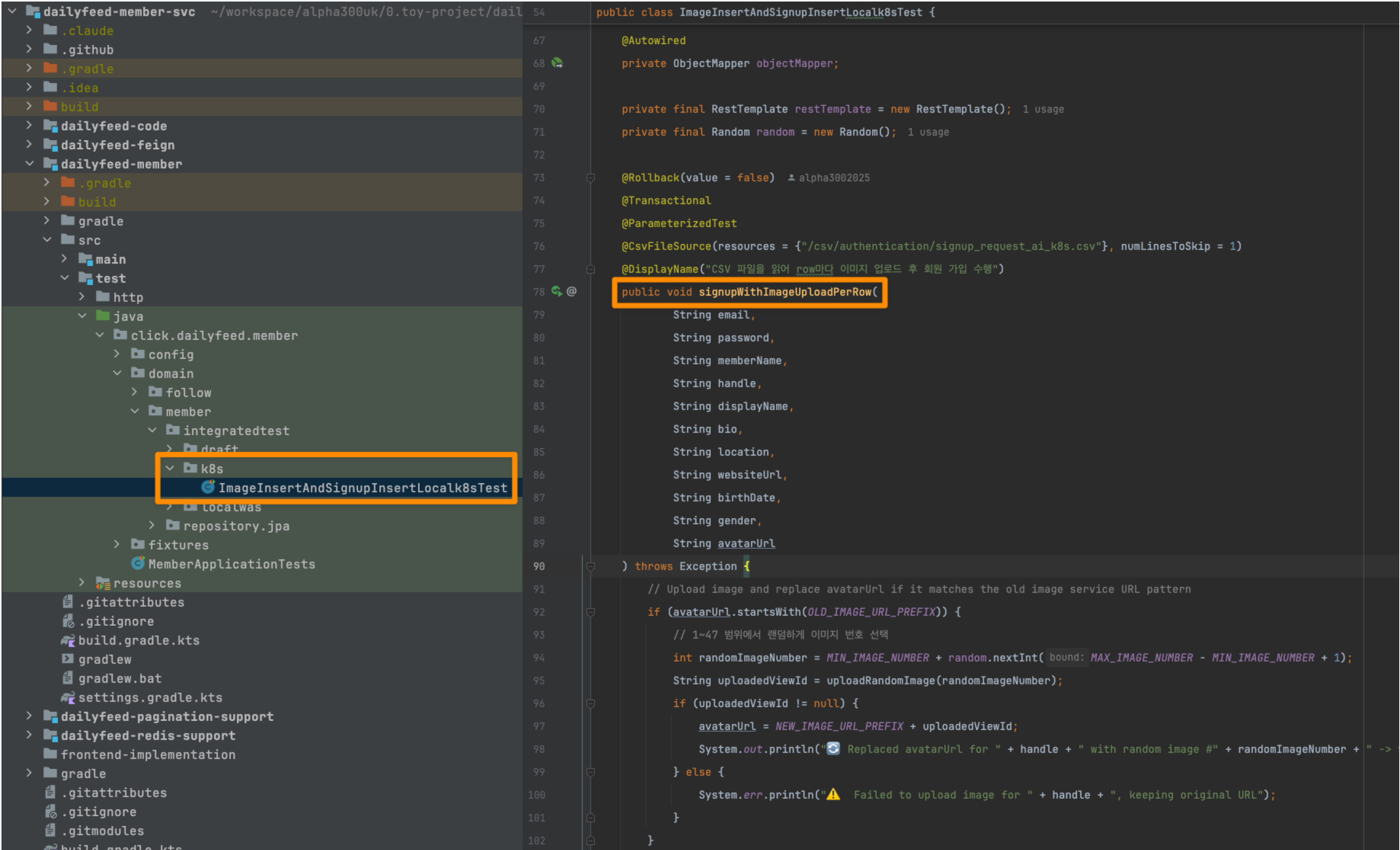


# 테스트 회원 데이터 insert (local-k8s)

회원을 50 명 정도를 직접 사람이 일일이 가입해서 follow/unfollow 하는 것을 테스트하기는 쉽지 않습니다.

이런 이유로 Test 코드로 테스트 데이터를 insert 하는 코드를 만들어두었습니다.

(2) local-k8s 프로파일 (로컬에서 kind 클러스터로 인프라(mysql,redis,kafka) 설치한 환경에서 deployment 로 각 리소스들을 구동했을때)



src/test/java/click.dailyfeed.member/domain/  
member/integratedtest/k8s/  
ImageInsertAndSignupInsertLocalk8sTest.java 를 선택합니다.

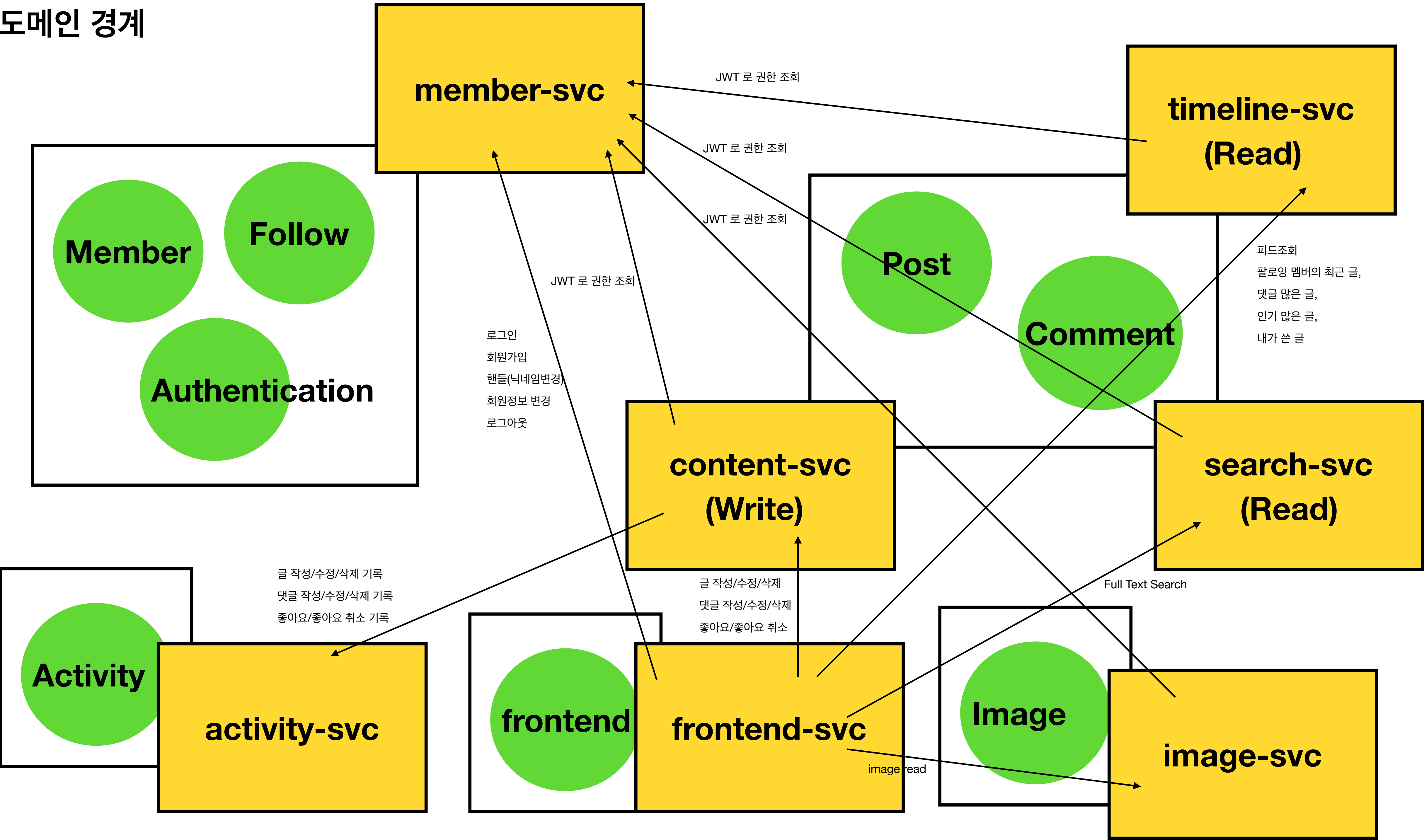
테스트 클래스 내에 다음 테스트 메서드를 실행합니다.

signupWithImageUploadPerRow()

이렇게 하면 k8s 클러스터 내의 infra (mysql) 에 회원 데이터  
들이 insert 됩니다.

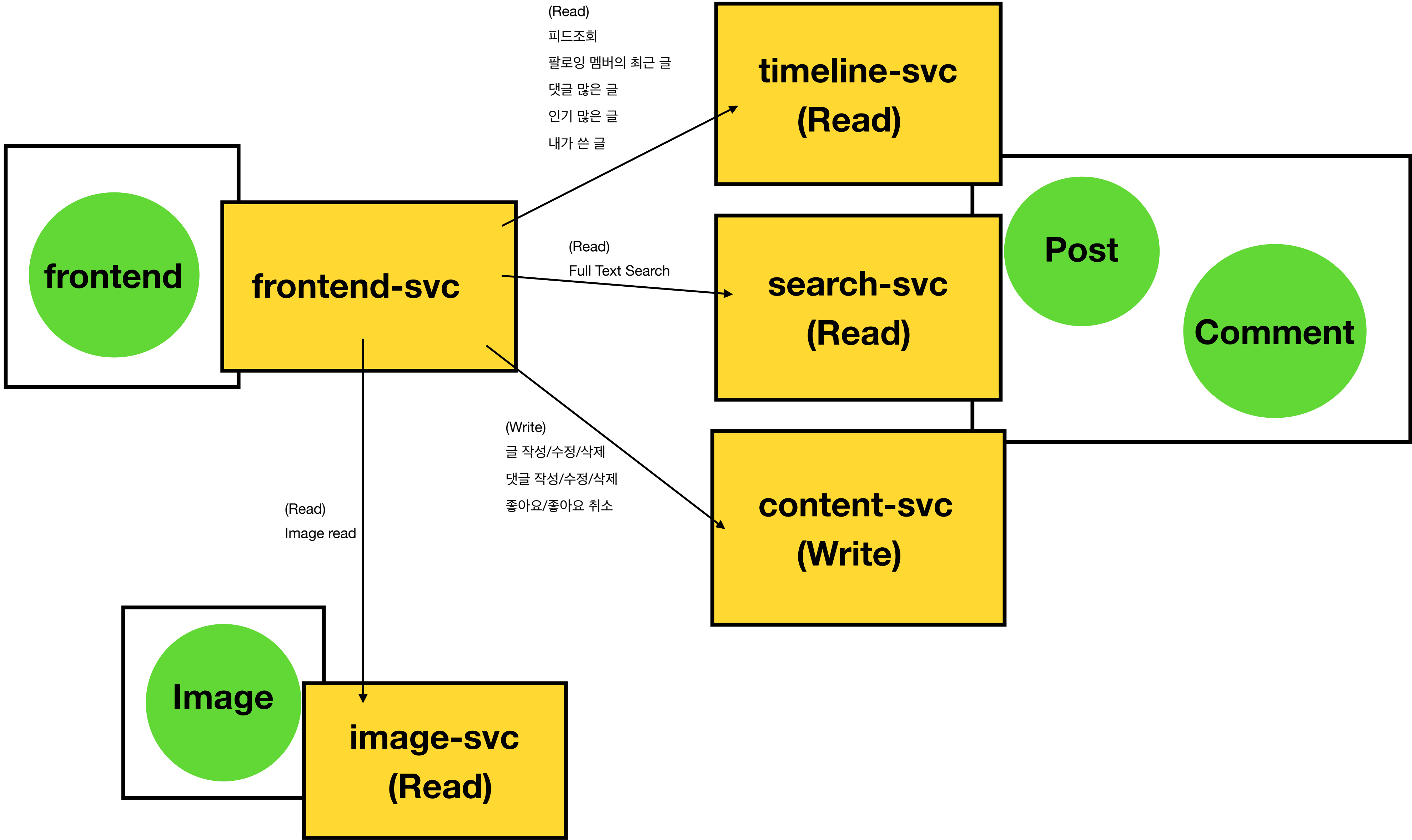
전반적인 흐름

도메인 경계

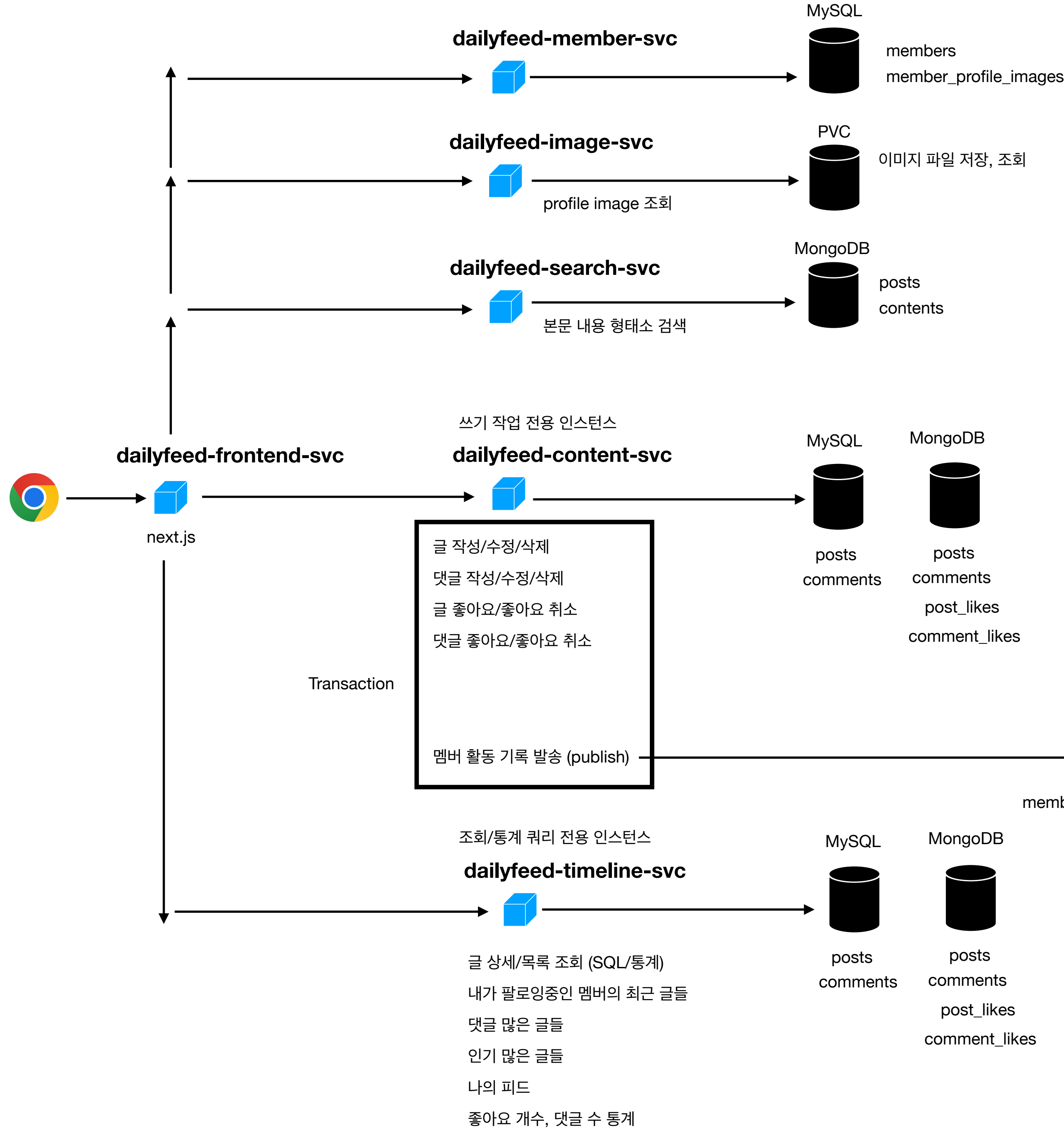




도메인 경계



# 전반적인 흐름



frontend 는 content, timeline search, image 서비스 각각에 요청을 보낼때

요청 헤더에 JWT 를 담아서 요청을 합니다.

content, timeline, search, image 서비스 각각은 요청 헤더 JWT 를 담아서

member-svc 에 실제 존재하는 사용자인지, 인증된 사용자인지 등을 조회합니다.

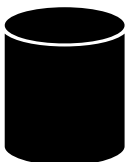
member-svc 에 해당 사용자가 인증된 사용자인지, 로그아웃했는지, 만료된 토큰을 가지고 있는지 등을 조회해서 Member에 대한 정보를 Profile 또는 Summary 등으로 return 해줍니다.

여기에 대해서는 뒤에서 자세히 설명합니다.



kafka

dailyfeed-activity-svc



member\_activities  
(MongoDB)

member\_activity-{yyyyMMdd}

**참고사항**

**dailyfeed-activity-svc**



# dailyfeed-activity-svc

## dailyfeed-activity-svc 관련

+ dailyfeed-activity-svc 는 kafka 사용시 이런 구조로 사용할 것이라는 예시를 남기기 위해 작성한 예제입니다.

+ 사용자가 서비스에 인입해서 어떤 활동을 했는지를 기록하는 서비스인데, 현존 플랫폼 들 중 ‘알림 배너’ 라는 기능에 이웃/친구 관계의 멤버가 어떤 활동(좋아요/좋아요취소/글작성/글수정)을 했는지를 보여주는 기능이 있는데, 이 기능에 대해 어떤 식으로 기록을 할지에 대한 예제 프로젝트입니다.

+ 만약 블로그 서비스가 있다고 해보겠습니다. 블로그 서비스의 운영 년수가 오래되어가고, 중요한 사업아이템이 되었을 때 블로그 서비스에서 글을 작성할 때 사용자의 활동 기록까지 수정하기에는 활동 기록 시에 필요한 부수적인 작업으로 블랙리스트 체크, 욕설 체크, 음란물 필터링, 광고 추천시스템 업데이트 등 복합적인 작업의 요구가 발생할 수 있는데 이 경우 글 쓰기/수정/삭제 기능에서 이 모든 기능을 수행하기 어려워집니다.

+ 이번 프로젝트에서는 이런 경우에 대해 활동기록, 블랙리스트 체크, 광고추천 시스템 업데이트 등의 작업을 dailyfeed-activity-svc 로 이관한 케이스를 가정합니다.

+ 그리고 사용자의 글 삽입/수정/삭제에 대해서는 POST\_CREATE, POST\_UPDATE, POST\_DELETE 이벤트를 발생시켜 Kafka Topic 으로 해당 이벤트를 발행하고, 이 것을 구독하는 dailyfeed-activity-svc 가 관련된 처리를 하도록 합니다. 즉, MSA 간 통신을 Kafka 를 통해 수행하는 과정을 예제의 전제조건으로 가정했습니다.

+ 카프카 운영시 카프카 증설 및 업그레이드 작업으로 인해 카프카 업그레이드 이슈 등이 있을 때는 세컨더리 저장소를 통해 Fail Over를 하는 것도 중요한 요소 중 하나입니다.

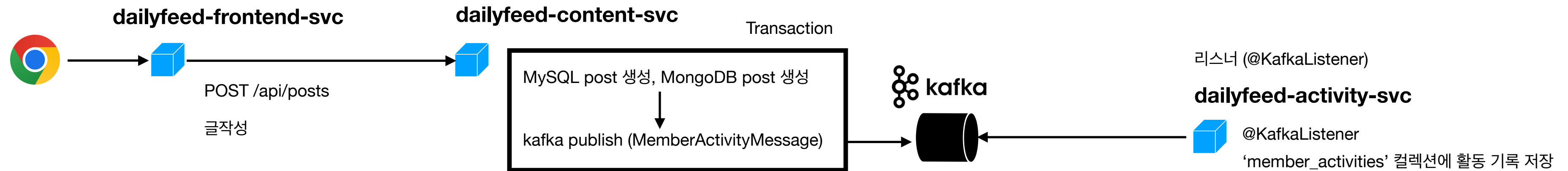
+ dailyfeed-content-svc 내에서 쓰기 작업 수행 도중 dailyfeed-activity-svc 가 장애가 나서 저장을 못하는 케이스 역시 가정합니다. dailyfeed-activity-svc 가 장애가 발생해도 dailyfeed-content-svc 의 글 쓰기/수정/삭제 작업에는 장애가 나지 않도록 하는 상황을 가정했습니다.

## **Case (1) Kafka**

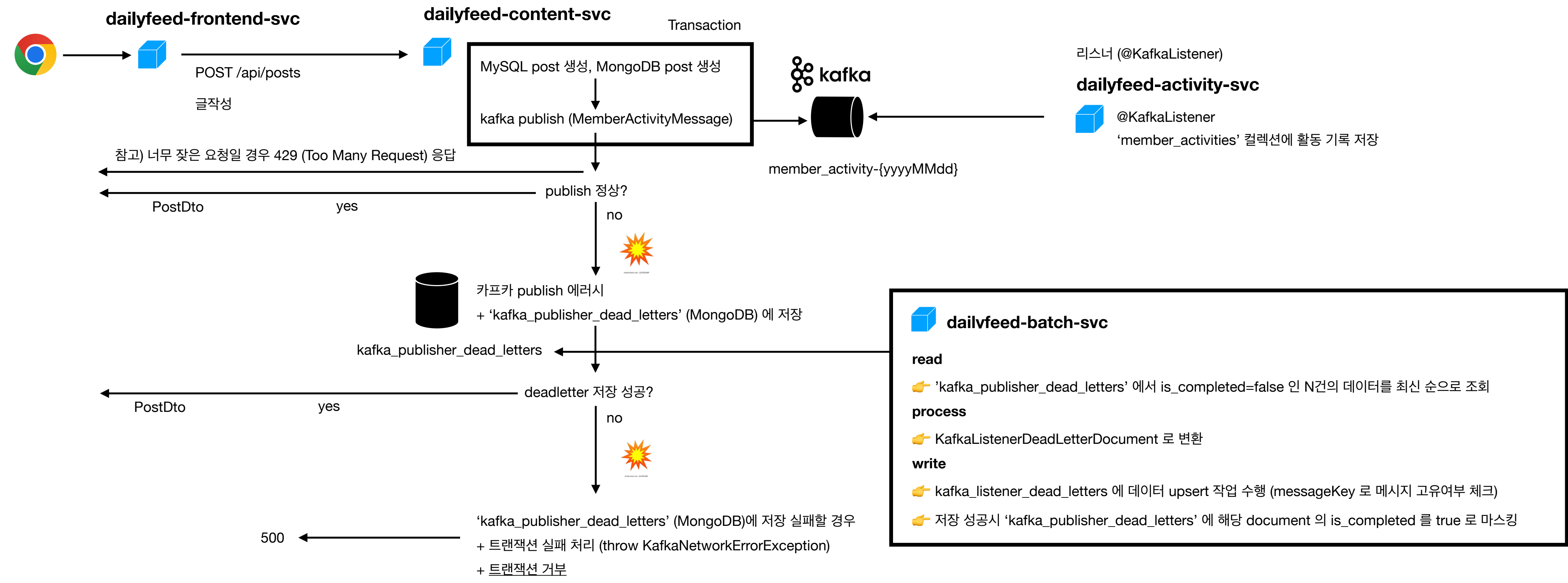
**: publisher, listener 데이터 처리 + 통신 에러 처리**



# Case (1) Kafka - 시나리오

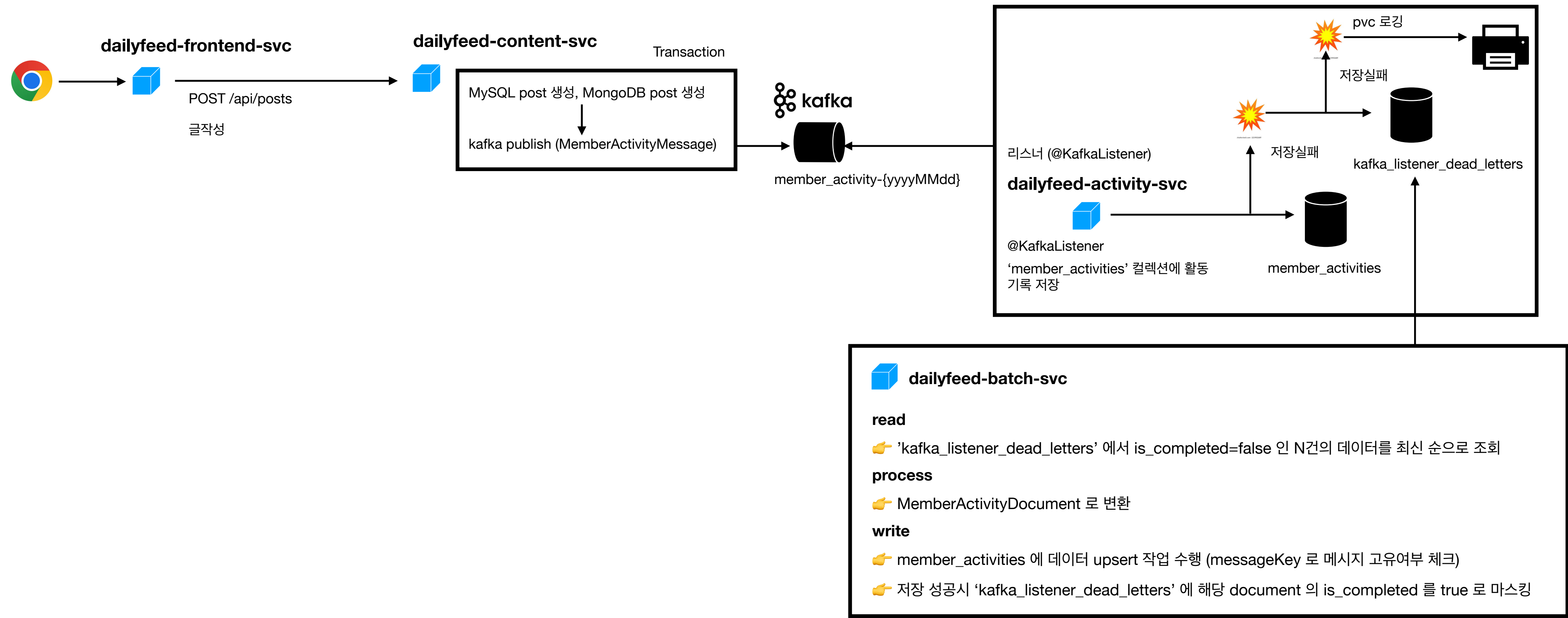


# Case (1) Kafka : publisher 측에서의 통신에러 처리



e.g.  
= 카프카 증설 및 운영에 용이하게 kafka 저장이 실패할 경우 deadletter 저장소에 저장 후 배치를 통해 발송하는 경우

# Case (1) Kafka : listener 측에서의 통신에러 처리

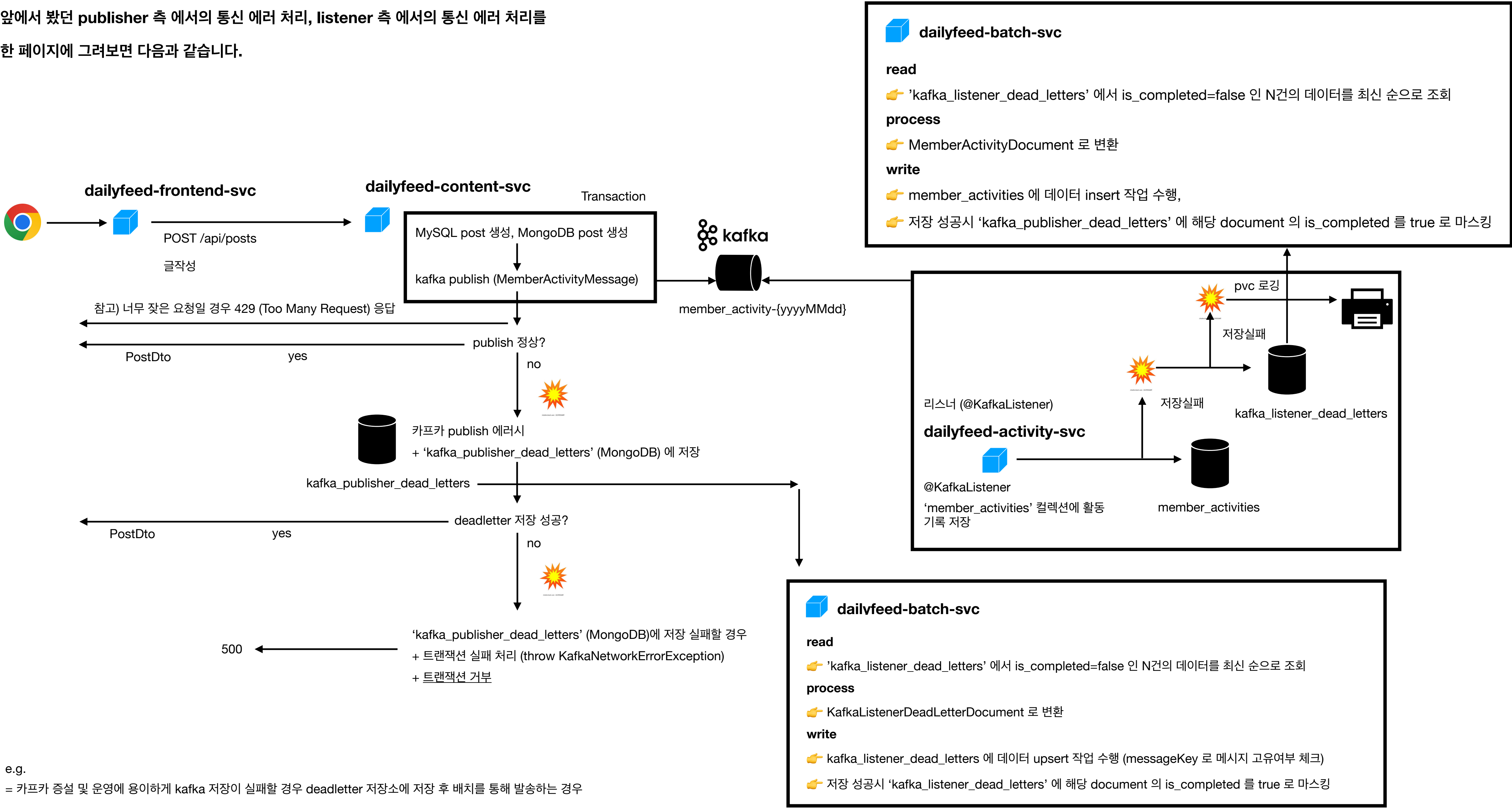


e.g.  
= 카프카 증설 및 운영에 용이하게 kafka 저장에 실패할 경우 deadletter 저장소에 저장 후 배치를 통해 발송하는 경우



# Case (1) Kafka : publisher,listener 측에서의 통신에러 처리

앞에서 봤던 publisher 측 에서의 통신 에러 처리, listener 측 에서의 통신 에러 처리를 한 페이지에 그려보면 다음과 같습니다.



e.g.  
= 카프카 증설 및 운영에 용이하게 kafka 저장이 실패할 경우 deadletter 저장소에 저장 후 배치를 통해 발송하는 경우

## **Case (2) Kafka**

**: Acks = 1, At Least Once**

**: 중복메시지 수신여부 체크 및 upsert**

# Case (2) - Acks = 1, At Least Once

## Producer Acknowledgement : Acks = 1 선택

참고)

**acks = 1** : Producer 가 메시지를 브로커에 보낼때 리더파티션에 메시지를 보낸 후, 리더파티션이 메시지를 받아서 로그에 쓴 후 리더로부터 ack 을 받는 방식입니다.

**acks = 0** : Producer 가 메시지를 브로커에 보낼때 메시지를 보내기만 하고 ack 를 기다리지 않는 방식입니다.

**acks = all (-1)** : Producer 가 메시지를 브로커에 보낼때 메시지를 보낸 후 리더파티션 & 모든 ISR 이 메시지를 받은 후 ack 을 받는 방식입니다.

(복제본에 모두 복제되어야 acks 를 받음)

acks = all 을 사용할 경우 min.insync.replicas=2 등으로 최소 한도의 복제 본 수를 지정해야 합니다.

현재 프로젝트에서는 일반적으로 많이 설정되는 Acks = 1 로 지정했습니다. Acks = 1 로 지정하면 리더가 죽고 팔로워가 복제를 받지 못한 경우 데이터가 손실될 가능성이 있지만 개발 버전의 환경상 많은 리소스가 불필요하고, 리더파티션 1기만 운영할 경우도 있기에 acks = 1 로 지정했습니다. 만약 acks=all 을 선택할 경우는 꼭 ‘min.insync.replcas’ 를 지정해야 합니다.

## Consumer Offset : At Least Once 선택

참고)

**At Most Once (최대 한번)** : Offset 을 먼저 커밋하고, 나중에 처리하는 방식입니다. 처리 중 실패할 경우 메시지가 손실됩니다. 최대 1번 처리됩니다.

메시지의 중복처리는 없지만, 데이터 손실가능성이 존재합니다.

**At Least Once (최소 한번)** : 처리를 먼저 하고 Offset을 나중에 커밋하는 방식입니다. 처리 후 커밋 전 실패할 경우 재처리 됩니다.(커밋이 안된 메시지는 재수신)

데이터의 손실은 없지만 메시지가 중복 수신 됩니다. 따라서 애플리케이션 레벨에서 메시지 중복 시에 대한 처리를 해주면 메시지 유실 없이 카프카 통신이 가능합니다.

가장 일반적으로 사용되는 방식입니다.

**Exactly Once (정확히 한번)** : 처리와 커밋이 하나의 트랜잭션으로 묶입니다. 둘 다 성공하거나 둘 다 실패합니다. 정확히 1번 처리됩니다.

중복처리가 없고 데이터 손실 역시 없는 방식이지만 성능 오버헤드가 있으며 구현이 복잡하고 추가설정이 필요합니다.

## dailyfeed 프로젝트

### + Producer Acknowledgement = Acks = 1 선택

: 리소스/비용을 줄이기 위해 리더 파티션 1기만 운영하는 경우까지 고려

### + Consumer Offset 커밋 방식 = At Least Once 선택

: 중복된 메시지를 받더라도 Redis 를 통해 중복 메시지를 체크하고, 데이터 저장시에도 같은 메시지일 경우 upsert 를 하도록 지정했습니다.



# Case (2) - 중복 메시지 체크 방식

메시지 중복체크

: 메시지 전송 시 메시지 키를 발급, 메시지 수신시 **Redis/Database** 에서 중복 수신 여부를 체크하는 방식을 사용

메시지 키 형식

POST\_CREATE

“member\_activity:kafka\_event:POST\_CREATE###{postId}###{memberId}”

POST\_UPDATE,POST\_DELETE,POST\_READ

“member\_activity:kafka\_event:{POST\_UPDATE|POST\_DELETE|POST\_READ}###{postId}###{memberId}###{yyyy-MM-dd HH:mm:ss.SSSSSSSSS}”

COMMENT\_CREATE

“member\_activity:kafka\_event:COMMENT\_CREATE###{postId}###{memberId}”

COMMENT\_UPDATE,COMMENT\_DELETE,COMMENT\_READ

“member\_activity:kafka\_event:{COMMENT\_UPDATE|COMMENT\_DELETE|COMMENT\_READ}###{commentId}###{memberId}###{yyyy-MM-dd HH:mm:ss.SSSSSSSSS}”

LIKE\_POST, LIKE\_POST\_CANCEL

“member\_activity:kafka\_event:{LIKE\_POST|LIKE\_POST\_CANCEL}###{postId}###{memberId}”

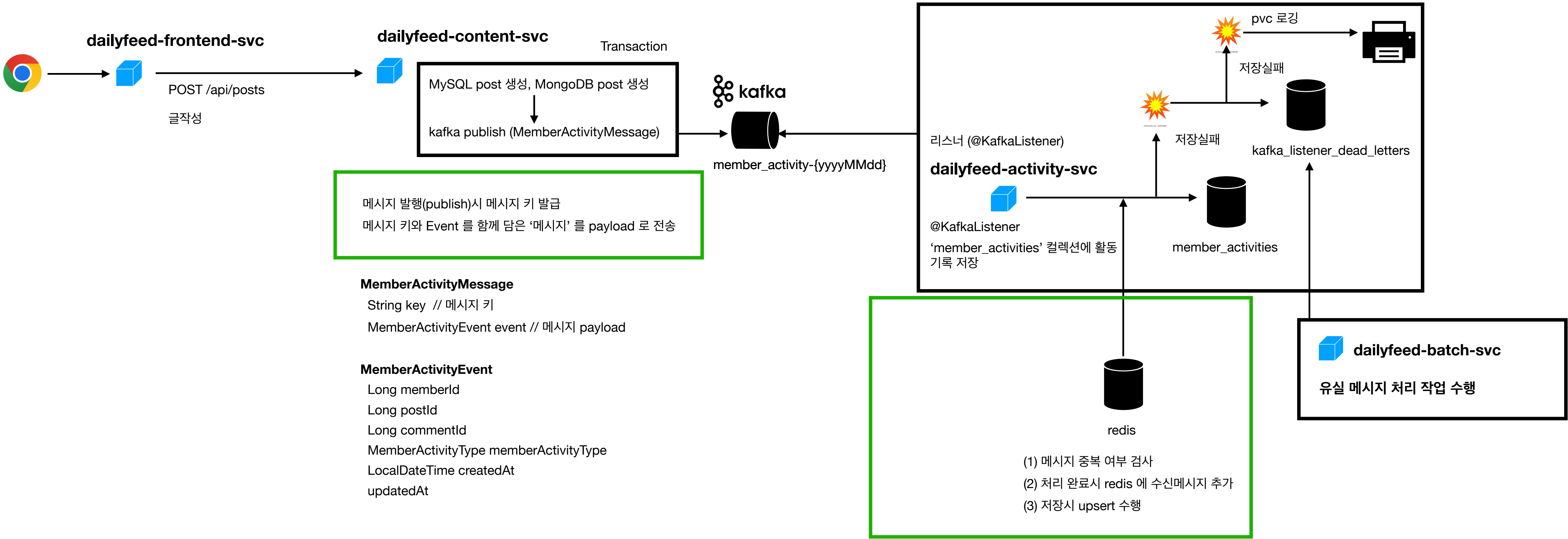
LIKE\_COMMENT, LIKE\_COMMENT\_CANCEL

“member\_activity:kafka\_event:{LIKE\_COMMENT|LIKE\_COMMENT\_CANCEL}###{commentId}###{memberId}”

# Case (2) - 중복 메시지 체크 방식

## 메시지 중복체크

: 메시지 전송 시 메시지 키를 발급, 메시지 수신시 Redis/Database 에서 중복 수신 여부를 체크하는 방식을 사용





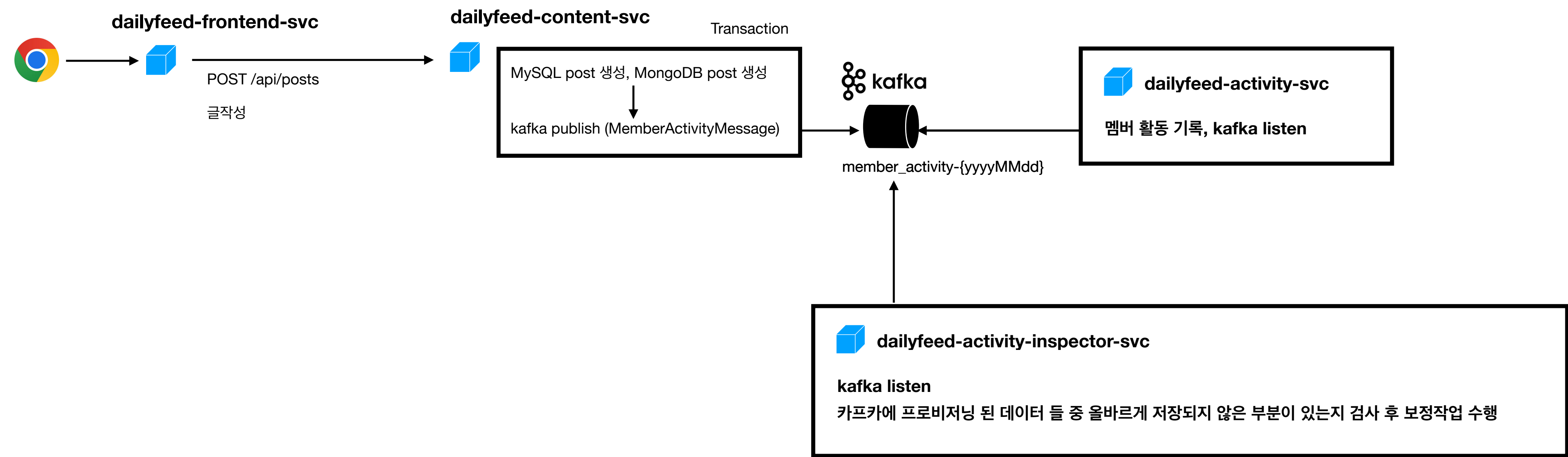
## Case (3) Kafka

: 날짜별 토픽 ( {topicName}-yyyyMMdd)

# Case (3) - 날짜별 토픽 ({topicName}-yyyyMMdd)

날짜별 토픽을 도입하게 된 이유

- : 데이터의 오류가 있는지 등에 대한 후보정 작업에 대해 유연하게 전략을 취할수 있다는 점
- : 이미 처리 완료된 데이터에 대한 토픽 (e.g. 7일 전)의 경우 토픽 삭제를 통해 운영시 카프카 브로커가 점유하는 디스크 사이즈를 줄일 수 있다는 점 (운영 비용 최적화 가능한 구조를 고려함)



## Case (3) Kafka

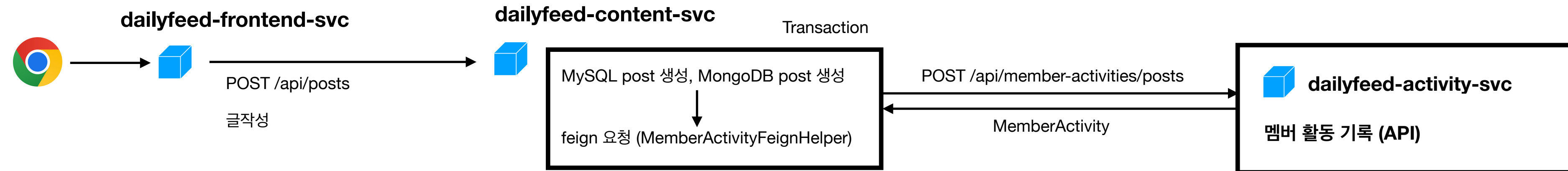
: Kafka → feign

publish-type 에 따른 kafka/feign 통신 선택

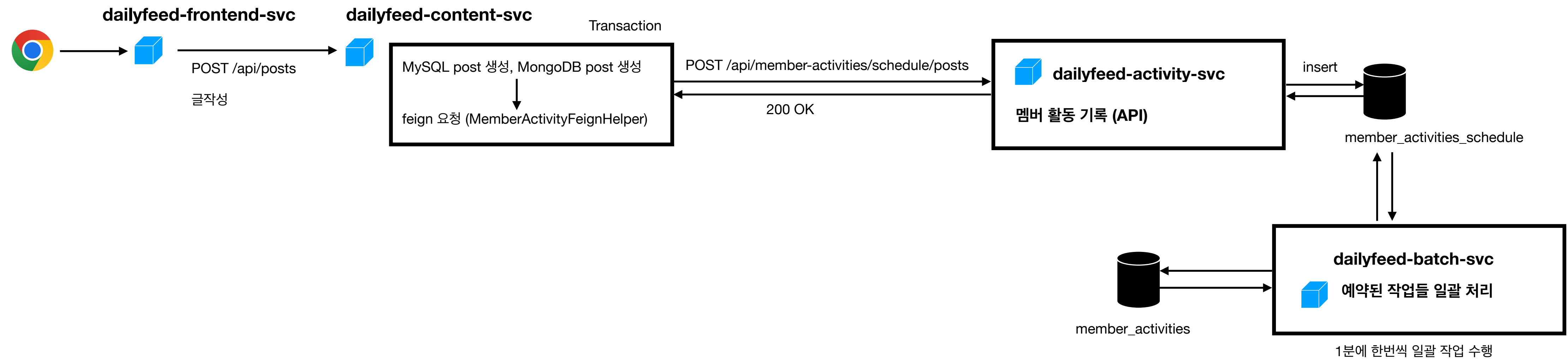


# Case (3) - Kafka → feign

동기적으로 구성할 경우 ( 'dailfyeed' 프로젝트의 현재 구성)



비동기적으로 구성할 경우



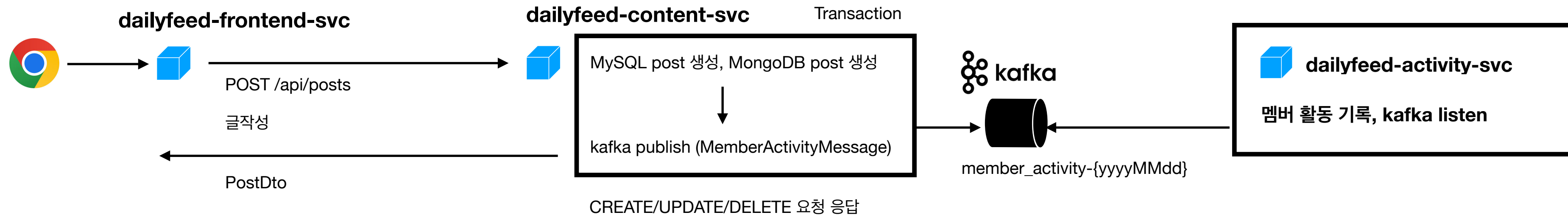
**Case (4) 스케일아웃 그룹 분류**

**: Read 스케일아웃 그룹**

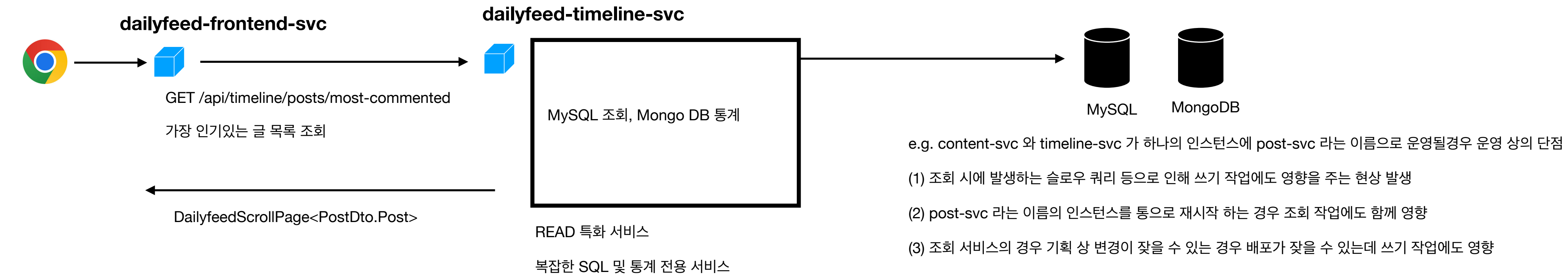
**: Create/Update/Delete 스케일아웃 그룹**

# Case (4) - Create/Update/Delete 트래픽과 Read 트래픽의 스케일아웃 그룹 분류

## Create/Update/Delete 트래픽



## Read 트래픽





## 참고) istio 란?

- 혹시라도 istio가 무엇인지 모르는 분들을 위한 설명 섹션입니다.

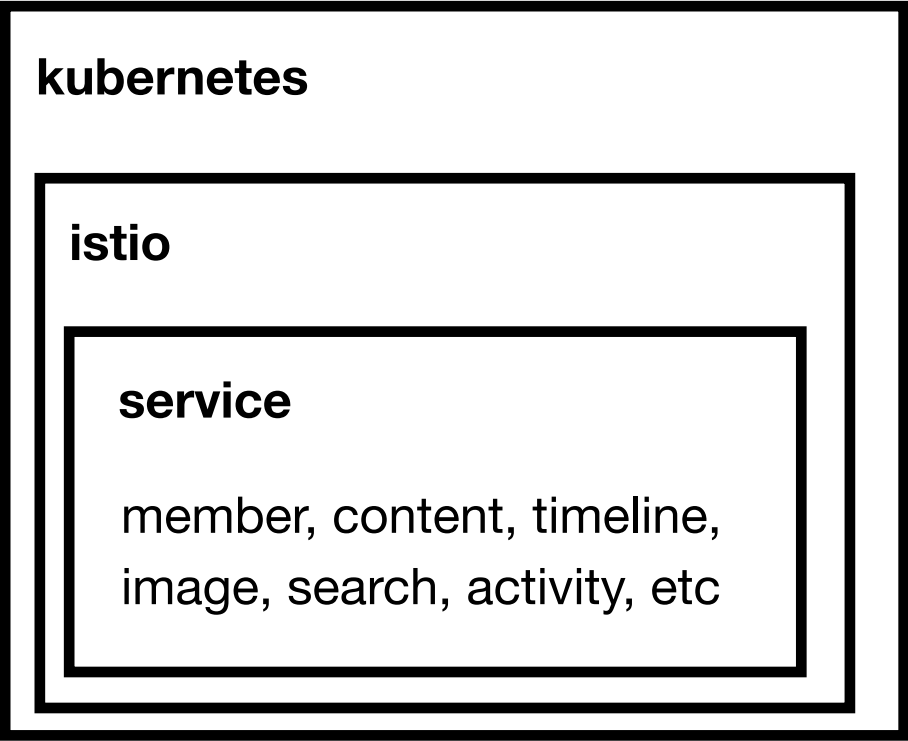
# Istio 란?

혹시라도 istio 가 무엇인지 모를 수도 있는 분들을 위해 간단한 설명을 추가합니다.

istio 란?



참고 : <https://istio.io>



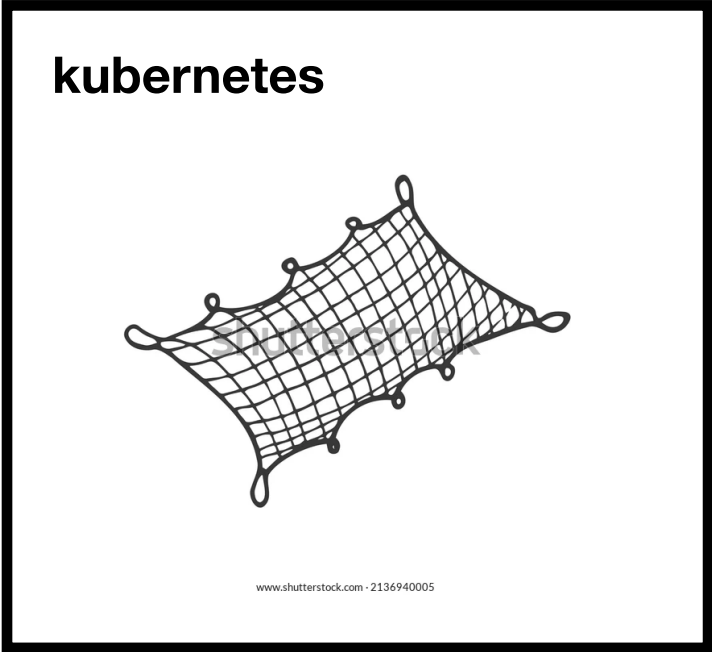
kubernetes 또는 여러 컨테이너 오케스트레이션 등에서 Envoy Proxy 역할을 수행하는 sidecar 를 통해

전체 네트워크의 서비스 메시 (Service Mesh) 레벨에서

라우팅, HTTPS 상호 보안, HttpRetry, Timeout, CircuitBreaker, HTTP Connection Pool 등의 기능을 제공하는

네트워크 유틸리티 플랫폼 입니다.

서비스 메시 (Service Mesh) 란?



Mesh 는 network 의 ‘그물’을 의미하는데, 이 ‘Service Mesh’ 라는 것은

kubernetes 의 Network 레벨을 의미하는 ‘Mesh’ 를 의미합니다.

Istio 를 사용하면, 1차적으로 ‘Mesh’라고 부르는 Network 레벨에서

통합적인 제어 (라우팅, HTTS, Timeout, HTTP Retry, CircuitBreaker, Connection Pool 등) 를 수행할 수 있습니다.

요청이 너무 많을 경우 그 요청을 모두 처리할 필요가 없을수도 있습니다.

너무 잦은요청은 네트워크 레벨에서 거부하고, 다음 타임윈도우에 재요청하게 하는 것이 나올 수 있습니다.

Service Mesh 는 이런 기능에서부터 여러가지 기능들을 제공합니다.

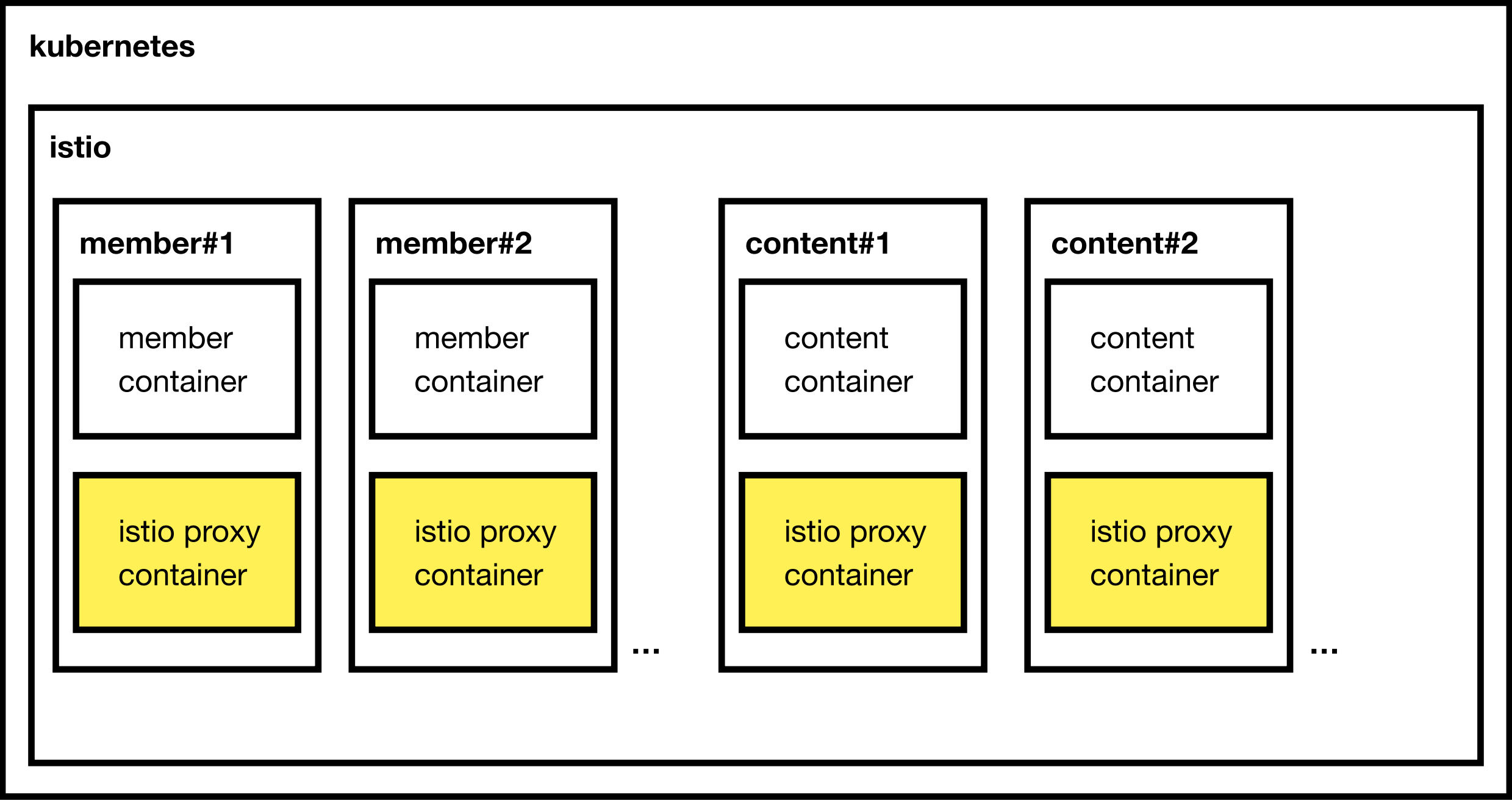
예를 들면 논리적인 라우팅을 통해 여러 버전의 인스턴스를 공존하게 하는 것 역시 가능합니다.



# Envoy Proxy 란 ?

혹시라도 istio 가 무엇인지 모를 수도 있는 분들을 위해 간단한 설명을 추가합니다.

## Envoy Proxy 란?



좌측 그림의 노란색 상자에는 ‘istio proxy container’ 라는 것이 각각 존재합니다.

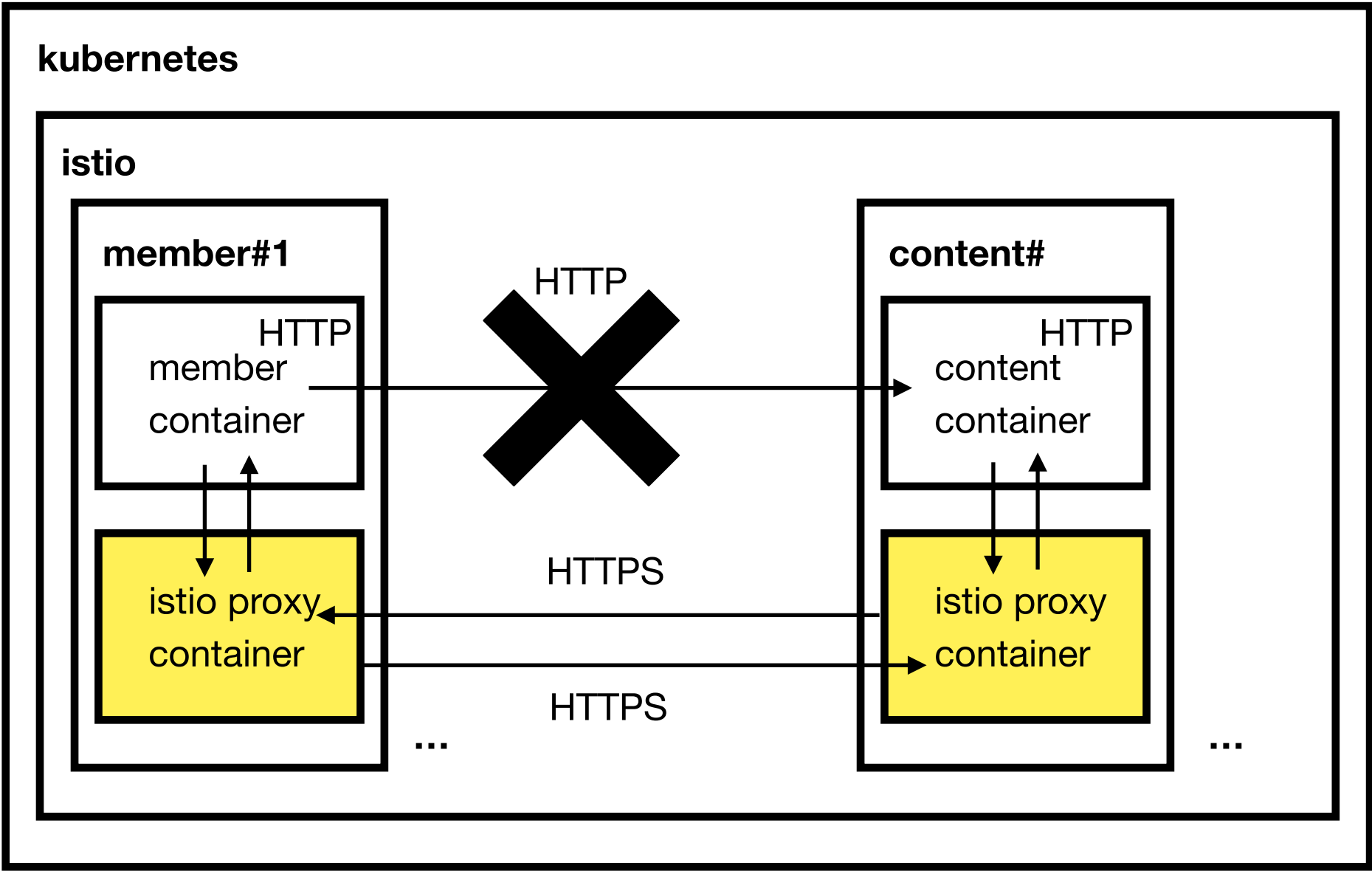
istio 는 이렇게 각각의 pod 내에 istio가 관리하는 container 를 주입해서 각각의 pod 에 대한 네트워크 제어를 수행합니다. 이렇게 주입된 ‘istio proxy container’ 들은 각각 자신의 파드를 대표해서 다른 서비스와 통신을 하는 ‘외교관’, ‘proxy’ 같은 역할을 수행합니다.

예를 들면 HTTPS 가 각각의 pod 에 적용이 안되어 있더라도 istio proxy container 가 주입되어 있고 PeerAuthentication 을 SRICT 로 설정해둔 상태라면, pod 간의 통신시 서비스 컨테이너가 HTTPS 요청을 하지 않더라도 istio proxy container 가 네트워크 레벨에서 HTTPS 암호화를 통해 파드간의 통신을 암호화합니다.

이렇게 각각의 파드에 대해 대사(Envoy) 역할을 하는 Proxy 를 Envoy Proxy 라고 일반적으로 부르며, 일반적으로는 Sidecar 라고 부르기도 하고, Proxy 라고 부르기도 하기도 합니다.

# Istio 사용의 장점 - (1) HTTPS

혹시라도 istio 가 무엇인지 모를 수도 있는 분들을 위해 간단한 설명을 추가합니다.



## (1) HTTPS (PeerAuthentication)

istio 에서는 deployment, pod 내에 배포한 container 가 기본적으로 HTTPS 를 적용하지 않았더라도 istio 의 proxy 는 기본적으로 pod 와 pod 간 통신을 HTTPS 로 암호화해서 통신할 수 있도록 네트워크 레벨에서 암호화를 처리해서 통신을 합니다.

이 설정은 PeerAuthentication 이라는 리소스를 통해 정의되며, 더 세부적인 AuthorizationPolicy 등을 적용하는 것 역시 가능합니다.

kubernetes 를 도입을 하더라도, 각각의 애플리케이션 서비스마다 HTTPS 인증서를 관리하는 것은 쉬운 일이 아닙니다. 만약 Istio 를 사용하게 된다면 이런 작업들을 PeerAuthentication 설정을 통해 최소화 할 수 있게 됩니다.

상황에 따라 pod 와 다른 pod 는 물리적으로 데이터 센터 중 다른 서버 건물 등에 배치될 수도 있습니다. 이 경우 외부와 통신할 때 HTTPS 가 아닌 HTTP 로 통신을 수행할 경우 통신 내용이 평문으로 전달되면서 탈취될 우려가 있는데, Istio 를 사용한다면 혹시라도 모를 1%의 유출 가능성 까지도 모두 암호화를 적용할 수 있다는 점은 장점입니다.

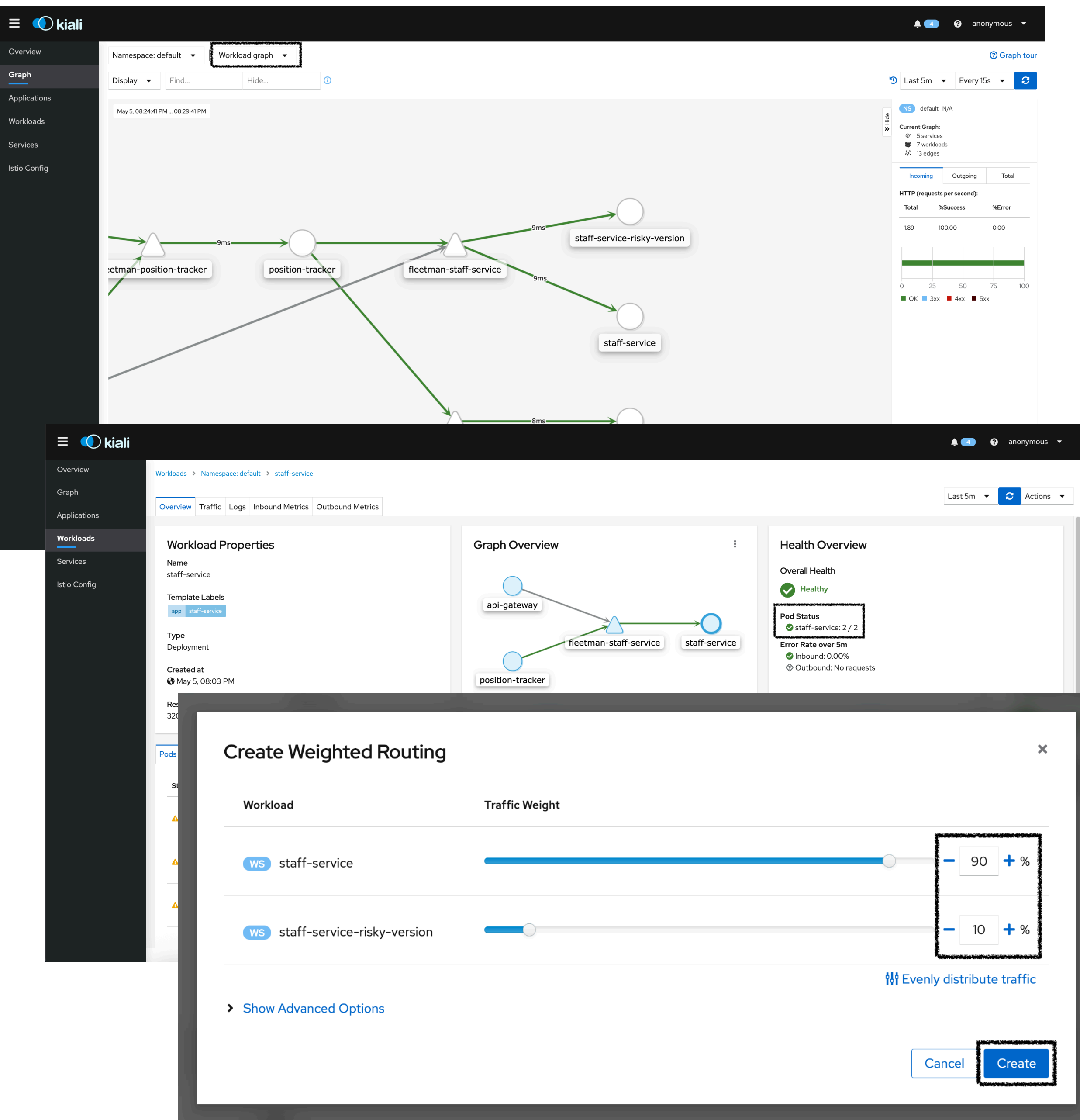
kubernetes 를 도입한다고 해도, istio 등에 대해서 잘 모른채로 부딪히기 식으로만 접근하면

결국 관리가 쉽지 않아질 가능성이 있습니다.

운영 레벨에서 kubernetes 를 제어하기 위해서는 최소한 istio 에 대한 관리 능력 정도는 필요하다고 생각합니다.

# Istio 사용의 장점 - (2) Kiali, virtual service, routing

혹시라도 istio 가 무엇인지 모를 수도 있는 분들을 위해 간단한 설명을 추가합니다.



## (2) kiali

istio 에서는 kiali 를 별도의 addon 으로 제공하는데, 각각의 서비스가 어디로 향하는지를 시각화해서 표현한 대시보드를 제공합니다. 이 대시보드 내에서 긴급상황 등에 대해 트래픽의 제어를 할수 있고, 전체 트래픽의 흐름을 확인하는 것 역시 가능합니다.

예를 들어 timeline 애플리케이션의 v2 버전의 디플로이먼트가 슬로우쿼리로 인해 장애가 나고 있다고 해보겠습니다. 이에 대응하기 위해 kiali 대시보드에서는 다음의 대응을 취할수 있습니다.

- v1 의 timeline 디플로이먼트를 배포
- timeline-svc 서비스로의 요청에 대해 v2 버전의 timeline 디플로이먼트로는 요청을 끊기
- timeline-svc 서비스로의 요청에 대해서는 앞으로는 v1 버전의 디플로이먼트로 트래픽이 향하도록 제어

물론 이런 기능은 kiali 에서만 제공되는 기능은 아닙니다. istio 의 yaml 편집을 통해서도 제공할 수 있으며 git 등의 버전관리 등이 적용되도록 일반적인 운영상황에서는 yaml 편집으로 인프라에 대한 내력을 관리합니다.

위와 같은 장애 상황에서만 사용하지는 않습니다. 신규버전 배포시 카나리 배포와 유사한 방식을 istio 내에서 적용할 수 있습니다. 예를 들면 다음과 같은 방식입니다.

- v2 버전 배포
- virtual service 는 5% 의 트래픽은 v2 를 라우팅, 95% 의 트래픽은 v1 을 라우팅
- (1일 후) virtual service 는 10% 의 트래픽은 v2 를 라우팅, 90% 의 트래픽은 v1 을 라우팅
- (7일 후) virtual service 는 50% 의 트래픽은 v2 를 라우팅, 50% 의 트래픽은 v1 을 라우팅
- (30일 후) virtual service 는 100% 의 트래픽은 v2 를 라우팅, 0% 의 트래픽은 v1 을 라우팅

이렇게 istio 를 사용하면 각 네트워크 트래픽에 weight 을 부여해서 virtual service 내에서 각 destination 에 대해 weight 을 부여해서 몇 퍼센트의 네트워크를 v2 로 흘려보내서 장애가 나는지 아닌지를 미리 파악해보면서 천천히 배포해볼수 있게 됩니다.



# Istio 사용의 장점 - (3) Destination Rule - CircuitBreaker, OutlierDetection

혹시라도 istio 가 무엇인지 모를 수도 있는 분들을 위해 간단한 설명을 추가합니다.

```
1  apiVersion: networking.istio.io/v1beta1
2  kind: DestinationRule
3  metadata:
4    name: dailyfeed-activity
5    namespace: dailyfeed
6  spec:
7    host: dailyfeed-activity
8    trafficPolicy:
9      # Load balancing: 최소 요청 수를 가진 인스턴스로 라우팅 (기본 ROUND_ROBIN보다 효율적)
10     loadBalancer:
11       simple: LEAST_REQUEST
12       leastRequestLbConfig:
13         choiceCount: 2 # 2개 인스턴스 중 최소 요청 선택
14
15     # Connection Pool Settings
16     connectionPool:
17       tcp:
18         maxConnections: 100 # TCP 최대 연결 수
19         connectTimeout: 3s # 연결 타임아웃
20         tcpKeepalive: # TCP Keepalive 설정
21           time: 7200s # 2시간
22           interval: 75s
23           probes: 9
24       http:
25         http1MaxPendingRequests: 100 # HTTP/1.1 대기 요청 수
26         http2MaxRequests: 500 # HTTP/2 최대 요청 수
27         maxRequestsPerConnection: 10 # 연결당 최대 요청 수 (연결 재사용 제한)
28         maxRetries: 3 # 최대 재시도 수
29         idleTimeout: 30s # 유휴 연결 타임아웃
30         h2UpgradePolicy: UPGRADE # HTTP/2 업그레이드 허용
31
32     # Outlier Detection (Circuit Breaker)
33     outlierDetection:
34       consecutive5xxErrors: 5 # 연속 5xx 에러 5회 시 제거
35       consecutiveGatewayErrors: 3 # 연속 게이트웨이 에러 3회 시 제거 (더 민감하게)
36       interval: 1m # 분석 간격
37       baseEjectionTime: 5m # 기본 제거 시간
38       maxEjectionPercent: 50 # 최대 제거 비율 (50% 이상 제거 방지)
39       minHealthPercent: 30 # 최소 건강한 인스턴스 비율 (30% 보장)
40       splitExternalLocalOriginErrors: true # 외부/내부 오류 구분
41
42     # Subsets (버전별 라우팅용 - 필요시 사용)
43     subsets:
44     - name: v1
45       labels:
46         version: v1
47
```

## (3) DestinationRule

Destination Rule 은 VirtualService 가 관리하는 목적지 룰을 의미하는데, 이 Destination Rule 은 Connection Pool 의 갯수부터 어떤 에러가 나타났을 때 잠시 Eject 할지, 얼마 동안 Eject 할지 등을 의미하는 Circuit Breaking, LoadBalaner 기능 까지도 지원합니다.

더 자세한 내용은 좌측의 캡처를 참고해주시기 바랍니다.

애플리케이션 레벨에서도 CircuitBreaker 를 제공하는 것 역시 가능합니다.

하지만 혹시라도 모르는 장애 상황에서 CircuitBreaker 가 적용되지 않은 애플리케이션 서비스는 있을수도 있습니다. 항상 모든 애플리케이션에 CircuitBreaker 를 적용할 수 있는 것은 아니기에, 이런 경우에 대해 네트워크 레벨에서 CircuitBreaker 가 작동된다면, 전체 서비스로 장애가 전파되지 않을 수 있다는 안전 대책을 마련할 수 있게 됩니다.

# istio 도입에 대해

혹시라도 istio 가 무엇인지 모를 수도 있는 분들을 위해 간단한 설명을 추가합니다.



위에서 살펴봤던 장점들은 istio의 모든 장점을 설명하지는 못합니다.

istio 도입은 필수가 아니지만, 아마도 대부분의 **kubernetes** 환경에서 제대로된 데브옵스 팀이 운영되고 있다면

아마도 대부분이 istio 를 운영하고 있을 가능성이 큼니다.

쿠버네티스 도입이 배포/장애 관리 등의 이슈로 인해 오히려 운영의 효율성을 떨어뜨릴때도 있는데

이 경우 istio 를 도입한다면 운영상의 어려움을 경감시킬 수 있습니다.

# **Case (5) Istio VirtualService, DestinationRule**

**: VirtualService**

**: DestinationRule**



# Case (5) - VirtualService

VirtualService 는 HttpTimeout, HttpRetry 를 정의

|                               | member                                                                                   | content                                                                                  | timeline                                                                                 | image                                                          | search                                                                                   | activity                                                                                 |
|-------------------------------|------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|----------------------------------------------------------------|------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| timeout                       | 10s                                                                                      | 10s                                                                                      | 10s                                                                                      | 30s<br>(이미지 처리는 시간이 오래 걸릴수도<br>있다는 점을 감안)                      | 15s<br>(검색은 조금 더 긴 타임아웃 필요)                                                              | 10s                                                                                      |
| retries/attempts              | 5                                                                                        | 5                                                                                        | 5                                                                                        | 3<br>(이미지 업로드는 재시도를 적게하기<br>위해)                                | 5                                                                                        | 5                                                                                        |
| retries/perTryTimeout         | 3s                                                                                       | 3s                                                                                       | 3s                                                                                       | 10s<br>(이미지 처리 시간을 고려)                                         | 4s                                                                                       | 3s                                                                                       |
| retries/retryOn               | gateway-error,<br>connect-failure,<br>refused-stream,<br>5xx,<br>retriable-4xx,<br>reset | gateway-error,<br>connect-failure,<br>refused-stream,<br>5xx,<br>retriable-4xx,<br>reset | gateway-error,<br>connect-failure,<br>refused-stream,<br>5xx,<br>retriable-4xx,<br>reset | gateway-error,<br>connect-failure,<br>refused-stream,<br>reset | gateway-error,<br>connect-failure,<br>refused-stream,<br>5xx,<br>retriable-4xx,<br>reset | gateway-error,<br>connect-failure,<br>refused-stream,<br>5xx,<br>retriable-4xx,<br>reset |
| retries/retryRemoteLocalities | true                                                                                     | true                                                                                     | true                                                                                     | true                                                           | true                                                                                     | true                                                                                     |

# Case (5) - DestinationRule

**Destination Rule** 은 **connectionPool**, **outlierDetection** 을 정의함

## member

**connectionPool**  
maxConnections: 100, connectionTimeout: 3s, tcpKeepalive (time = 7200s, interval = 75s, probes = 9), http1MaxPendingRequests:100, http2MaxRequests:500, maxRequestsPerConnection:10, maxRetries:3, idleTimeout:30s, h2UpgradePolicy: UPGRADE

**outlierDetection**  
consecutive5xxErrors: 5, consecutiveGatewayErrors: 3, interval: 1m, baseEjectionTime: 5m, maxEjectionPercent: 50, minHealthPercent: 30, splitExternalLocalOriginErrors: true

## content

**connectionPool**  
maxConnections: 100, connectionTimeout: 3s, tcpKeepalive (time = 7200s, interval = 75s, probes = 9), http1MaxPendingRequests:100, http2MaxRequests:500, maxRequestsPerConnection:10, maxRetries:3, idleTimeout:30s, h2UpgradePolicy: UPGRADE

**outlierDetection**  
consecutive5xxErrors: 5, consecutiveGatewayErrors: 3, interval: 1m, baseEjectionTime: 5m, maxEjectionPercent: 50, minHealthPercent: 30, splitExternalLocalOriginErrors: true

## timeline

**connectionPool**  
maxConnections: 100, connectionTimeout: 3s, tcpKeepalive (time = 7200s, interval = 75s, probes = 9), http1MaxPendingRequests:100, http2MaxRequests:500, maxRequestsPerConnection:10, maxRetries:3, idleTimeout:30s, h2UpgradePolicy: UPGRADE

**outlierDetection**  
consecutive5xxErrors: 5, consecutiveGatewayErrors: 3, interval: 1m, baseEjectionTime: 5m, maxEjectionPercent: 50, minHealthPercent: 30, splitExternalLocalOriginErrors: true

## search (검색 특성상 pending 에 대해 허용치를 조금 크게 지정)

**connectionPool**  
maxConnections: 120, connectionTimeout: 3s, tcpKeepalive (time = 7200s, interval = 75s, probes = 9), http1MaxPendingRequests:120, http2MaxRequests:550, maxRequestsPerConnection:10, maxRetries:3, idleTimeout:45s, h2UpgradePolicy: UPGRADE

**outlierDetection**  
consecutive5xxErrors: 5, consecutiveGatewayErrors: 3, interval: 1m, baseEjectionTime: 5m, maxEjectionPercent: 50, minHealthPercent: 30, splitExternalLocalOriginErrors: true

## image (이미지 특성상 pending 에 대해 허용치를 크게 지정)

**connectionPool**  
maxConnections: 150, connectionTimeout: 3s, tcpKeepalive (time = 7200s, interval = 75s, probes = 9), http1MaxPendingRequests:150, http2MaxRequests:600, maxRequestsPerConnection:5, maxRetries:2, idleTimeout:60s, h2UpgradePolicy: UPGRADE

**outlierDetection**  
consecutive5xxErrors: 5, consecutiveGatewayErrors: 3, interval: 1m, baseEjectionTime: 5m, maxEjectionPercent: 50, minHealthPercent: 30, splitExternalLocalOriginErrors: true

## activity

**connectionPool**  
maxConnections: 100, connectionTimeout: 3s, tcpKeepalive (time = 7200s, interval = 75s, probes = 9), http1MaxPendingRequests:10, http2MaxRequests:500, maxRequestsPerConnection:10, maxRetries:3, idleTimeout:30s, h2UpgradePolicy: UPGRADE

**outlierDetection**  
consecutive5xxErrors: 5, consecutiveGatewayErrors: 3, interval: 1m, baseEjectionTime: 5m, maxEjectionPercent: 50, minHealthPercent: 30, splitExternalLocalOriginErrors: true



# Case (6) HPA

# Case (6) - HPA란?

HPA 가 무엇인지 모를 수 있는 분들을 위해 간단한 설명을 정리합니다.

## HPA (Horizontal Pod Autoscaler)

```
➔ dailyfeed-installer git:(main) kubectl get po -n dailyfeed
NAME                                READY   STATUS    RESTARTS   AGE
dailyfeed-activity-69844c4499-5fktm 2/2     Running   0           4m29s
dailyfeed-activity-69844c4499-79lhg 2/2     Running   0           4m14s
dailyfeed-activity-69844c4499-gfhwh 0/2     Init:1/2   0           102s
dailyfeed-content-5f88cc6bb9-5hwx8 0/2     Pending   0           87s
dailyfeed-content-5f88cc6bb9-7km7n 2/2     Running   0           4m15s
dailyfeed-content-5f88cc6bb9-fbhkg 1/2     Running   2 (28s ago) 2m14s
dailyfeed-content-5f88cc6bb9-mwwwh 1/2     CrashLoopBackOff 2 (13s ago) 4m30s
dailyfeed-frontend-5f58b55f68-lgzr6 2/2     Running   0           4m28s
dailyfeed-frontend-5f58b55f68-rck8d 2/2     Running   0           4m28s
dailyfeed-image-79c6459d4b-fgpqd 2/2     Running   0           4m28s
dailyfeed-image-79c6459d4b-qcsc6 2/2     Running   0           4m28s
dailyfeed-image-79c6459d4b-t6846 2/2     Running   0           4m28s
dailyfeed-member-7d59798c58-7qs96 2/2     Running   0           4m15s
dailyfeed-member-7d59798c58-ddqpn 1/2     CrashLoopBackOff 1 (9s ago) 2m59s
dailyfeed-member-7d59798c58-hbxqg 1/2     Running   2 (24s ago) 2m29s
dailyfeed-member-7d59798c58-qkxrk 0/2     Init:1/2   0           102s
dailyfeed-member-7d59798c58-zt2gt 2/2     Running   0           4m30s
dailyfeed-search-5d668444cd-dztlf 2/2     Running   0           4m28s
dailyfeed-search-5d668444cd-k4pct 2/2     Running   0           4m13s
dailyfeed-timeline-84c67c7f89-8x6hk 1/2     CrashLoopBackOff 1 (16s ago) 2m13s
dailyfeed-timeline-84c67c7f89-9rsrb 2/2     Running   0           4m29s
dailyfeed-timeline-84c67c7f89-nn9wx 2/2     Running   0           3m14s
dailyfeed-timeline-84c67c7f89-pj5mn 2/2     Running   0           4m29s
dailyfeed-timeline-84c67c7f89-w7tj8 0/2     Init:1/2   0           16s
```

## metrics-server

```
➔ dailyfeed-installer git:(main) kubectl top nodes
NAME                                CPU(cores)   CPU(%)   MEMORY(bytes)   MEMORY(%)
istio-cluster-control-plane         140m         1%        819Mi           6%
istio-cluster-worker                450m         5%       4677Mi          38%
istio-cluster-worker2              260m         3%       4408Mi          36%
```

배포되어 있는 deployment 의 replicas 의 수를 조정하려면 어떻게 해야할까요?

일반적인 방법은 deployment 를 kubectl edit 을 통해 수정하면 됩니다. 하지만, 예상하지 못한 트래픽의 급증 상황이고 현재 중요한 회의에 참석 중이어서 대응을 못해서 애플리케이션으로의 요청이 계속 실패하거나 슬로우 쿼리 등으로 인해 애플리케이션의 성능이 계속 느려진다면 어떻게 될까요?

HPA 를 적용하면 이런 문제가 해결됩니다. Horizontal Scaling 을 Auto 로 하겠다는 의미의 Horizontal Pod Autoscaler 는 Pod 의 메트릭을 검사해서 특정 수치에 도달했을때 scale out 을 진행하게 됩니다. 사람이 수동으로 진행하는 것에 비해 동적으로 대응할 수 있다는 점에서 장점이 있습니다.

HPA 를 사용하기 위해서는 metrics-server 를 설치해야합니다.

HPA 는 pod 또는 node 의 상태 데이터 메트릭을 보고 사용자가 지정한 임계값에 따라 Auto Scaling 을 진행하기 때문입니다.

설치 후에는 kubectl top nodes, kubectl top pods -n {네임스페이스} 명령으로 관련 리소스들의 상태를 확인할 수 있습니다.

# Case (6) - 현재 HPA 설정

현재 지정한 HPA 설정을 설명합니다.

| 서비스          | Min/Max Replicas | CPU 임계값 | 메모리 임계값 | Scale Up 특징 | 비고             |
|--------------|------------------|---------|---------|-------------|----------------|
| activity-svc | 2-10             | 70%     | 90%     | 표준 (60초)    | 표준 설정          |
| content-svc  | 2-10             | 70%     | 90%     | 표준 (60초)    | 표준 설정          |
| frontend-svc | 2-15             | 60%     | 70%     | 즉시 (0초)     | 사용자 대면, 가장 공격적 |
| image-svc    | 2-15             | 65%     | 90%     | 표준 (60초)    | CPU/메모리 집약적    |
| member-svc   | 2-12             | 70%     | 90%     | 표준 (60초)    | 인증 가용성 중요      |
| search-svc   | 2-12             | 65%     | 90%     | 표준 (60초)    | 빠른 응답 필요       |
| timeline-svc | 2-10             | 70%     | 90%     | 표준 (60초)    | 표준 설정          |

## (1) 안정성 우선

- 모든 서비스에 대해 **최소 2개**의 replica 부여
- scale down 시 5분의 안정화 윈도우(stabilization window)를 부여해 급격한 scale down 방지

## (2) 서비스 특성 별 세분화

- frontend-svc : 사용자 경험을 위해 빠른 스케일 업 (스케일업 안정화 윈도우 : 0초)
- image-svc, search-svc : CPU 워크로드 임계값을 낮은 값을 부여

## (3) 점진적 스케일링

- 1분 마다 스케일업 검사, 1분마다 임계값에 도달시 2개, 또는 5개의 Pod(frontend)를 스케일업
- 1분마다 스케일다운 검사, 1분마다 임계값에 도달시 2개, 50% 축소

## (4) 리소스 requests, limits 세부 정의

- 리소스마다 top 명령을 통해 파악한 최소,최대의 범위를 대략적으로 파악 후 적용 (가장 오래 걸렸던 노가다 작업)



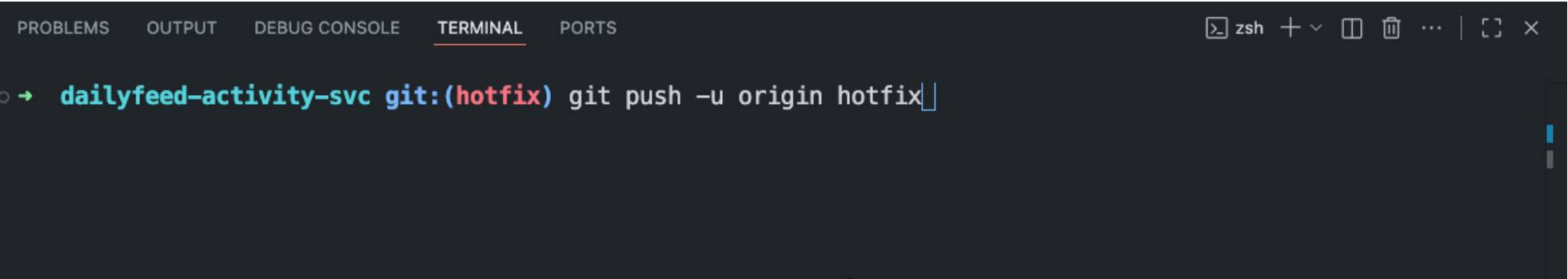
## Case (7) 도커 이미지 빌드 자동화 (github workflow)

: 자동 모드

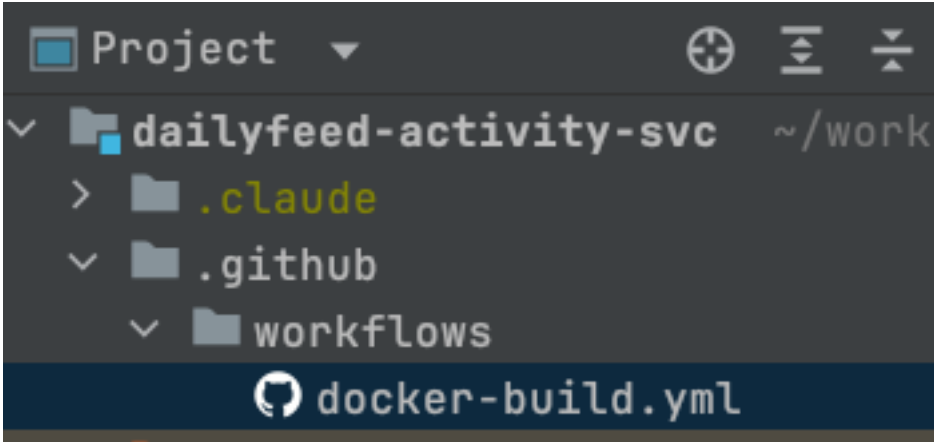
: 수동 모드

# Case (7) - 도커 이미지 빌드 자동화 (github workflow) - 자동 모드

workflow 대상 브랜치 : main/develop/hotfix



commit & push



github repository 내의 workflow 발동 (trigger)

참고 : <https://github.com/alpha3002025/dailyfeed-activity-svc/blob/main/.github/workflows/docker-build.yml>

이미지 태그 형식

: {branchName}-{yyyyMMdd}-{build번호}

← Docker Build and Push

fix: workflow modified (#6) #15

Summary

Jobs

build-and-push

Run details

Usage

Workflow file

Triggered via push 4 hours ago

alpha3002025 pushed · 9c2c7ef main

Status

Success

Total duration

2m 15s

Artifacts

—

docker-build.yml

on: push

build-and-push 2m 6s

build-and-push summary

Docker Build Summary 🚀

- Image: alpha300uk/dailyfeed-activity-svc
- Tag: main-20251023-2316
- Triggered by: alpha3002025
- Commit: 9c2c7efd546674d45c08d8145a70c745466a9ee7

| Gradle Root Project    | Requested Tasks           | Gradle Version | Build Outcome | Build Scan®   |
|------------------------|---------------------------|----------------|---------------|---------------|
| dailyfeed-activity-svc | :dailyfeed-activity:build | 8.14.3         | ✓             | Not published |
| dailyfeed-activity-svc | :dailyfeed-activity:jib   | 8.14.3         | ✓             | Not published |

docker 이미지 jib build & push

alpha300uk

Docker Personal

Repositories

Hardened Images NEW

Collaborations

Settings

Default privacy

Notifications

Billing

Usage

Pulls

Storage

Repositories / dailyfeed-activity-svc / General

alpha300uk/dailyfeed-activity-svc 🐙

Last pushed 41 minutes ago · Repository size: 1.4 GB

Add a description

Add a category

General

Tags

Image Management BETA

Collaborators

Webhooks

Settings

Tags

This repository contains 22 tag(s).

| Tag                  | OS | Type  | Pulled          | Pushed         |
|----------------------|----|-------|-----------------|----------------|
| latest               | 🐧  | Image | less than 1 day | 41 minutes     |
| main-20251024-0252   | 🐧  | Image | less than 1 day | 41 minutes     |
| main-20251023-2316   | 🐧  | Image | less than 1 day | about 4 hours  |
| hotfix-20251023-2315 | 🐧  | Image | less than 1 day | about 4 hours  |
| hotfix-20251023-0010 | 🐧  | Image | less than 1 day | about 19 hours |

See all

DOCKER SCOUT INACTIVE

Activate



# Case (7) - 도커 이미지 빌드 자동화 (github workflow) - 수동 모드

workflow 대상 브랜치 : main/develop/hotfix

## github workflow 실행

alpha3002025 / dailyfeed-activity-svc

CodeIssuesPull requestsActionsProjectsSecurityInsightsSettings

Actions

Docker Build and Push

16 workflow runs

submodule update

fix: workflow modified (#6)

fix: workflow modified

Docker Build and Push

Use workflow from

Branch: main

Docker image version (e.g., {branch name}-20251023-0001) \*

release-20251024

Run workflow

docker hub (push 된 이미지 확인)

My Hub

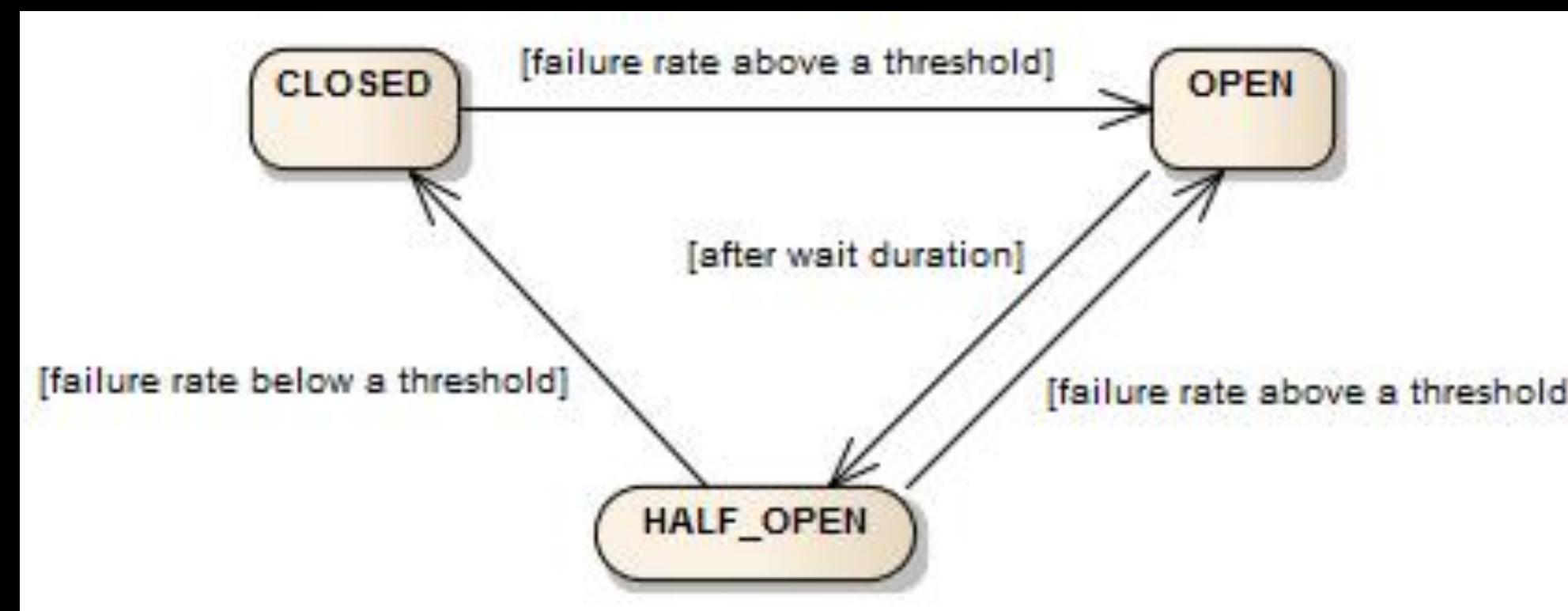
Search Docker Hub

alpha300uk/dailyfeed-activity-svc

Tags

| Tag                | OS | Type  | Pulled          | Pushed             |
|--------------------|----|-------|-----------------|--------------------|
| latest             |    | Image | less than 1 day | less than a minute |
| release-20251024   |    | Image | less than 1 day | less than a minute |
| main-20251024-0252 |    | Image | less than 1 day | about 1 hour       |

# Case (8) resilience4j feign : circuitbreaker, rate limiter



# Case (8) - circuit breaker

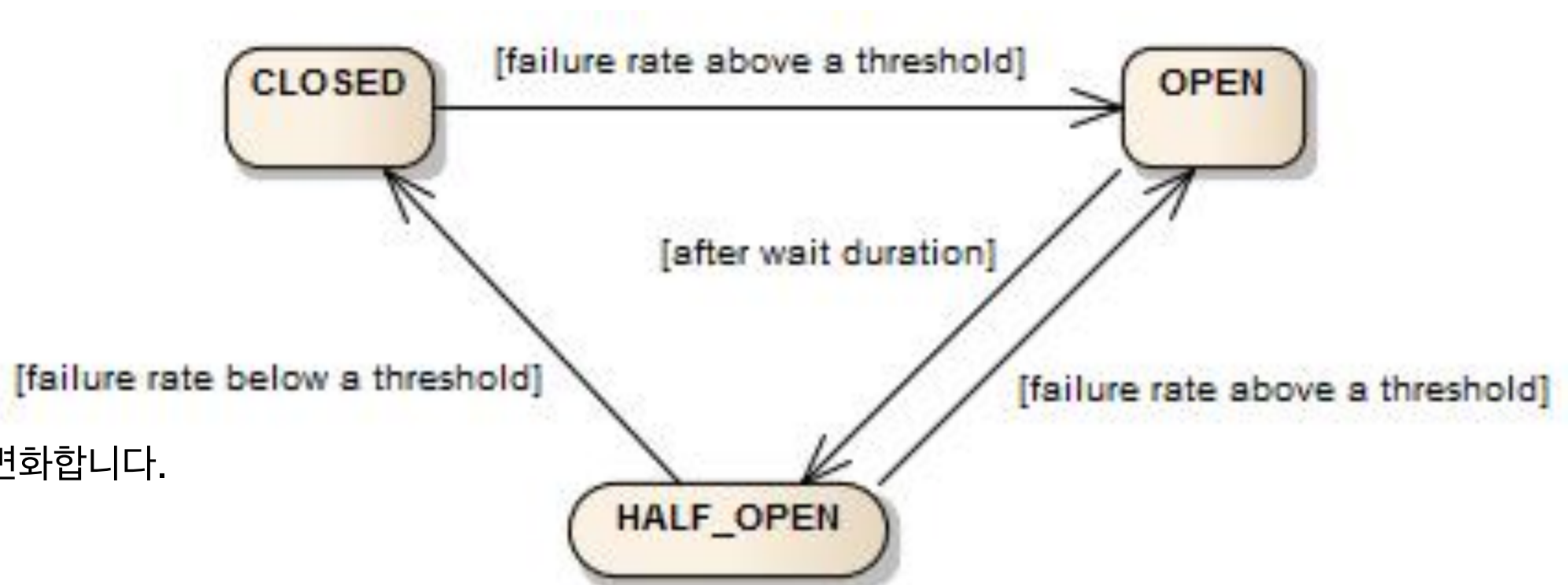
**circuit breaker** : 현재 서비스에서 다른 마이크로서비스로 **feign** 을 통해 요청을 호출시 장애가 났다고 판정을 하고 해당 서비스로의 호출을 차단하거나 자동복구되는 차단을 하거나 장애 회복시 복구를 하는 등의 설정을 제공하는 기능

## 장점

- 연쇄 장애(Cascading Failure) 방지, 빠른 실패 (Fail Fast), 자동 복구 (Self Healing)
- 사람이 즉시 대응할 수 없는 요소에 대해 장애를 감지해서 차단을 하거나 자동 복구가 되도록 지정 가능

## 주요 상태

- CLOSED (닫힘 - 정상), OPEN (열림 - 차단), HALF\_OPEN(반열림 - 테스트)
- HALF\_OPEN 시에 서비스 회복 여부를 체크하는 네트워크 테스트가 내부적으로 수행되며, 테스트 결과에 따라 CLOSED(정상복구), OPEN(차단)으로 변화합니다.



설정의 정도에 따라 **default**, **fast**, **conservative** 로 분류해두었고, 각 서비스별로 다음과 같이 지정했습니다.

critical : timelineService, memberService

default : contentService, activityService

bulk : imageService

다음은 **default** 에 대한 설정입니다.

max-attempts: 3

- 최대 3번까지 시도 (최초 1번 + 재시도 2번), 실패 시 총 3번의 호출 시도 후 최종 실패 처리

wait-duration: 500ms

- 각 재시도 사이의 대기 시간 (실패 후 재시도 하지않고 500ms 를 기다린 후 재시도)

## - 장점

- Backoff : 대상 서비스에 즉시 재시도하지 않고 일정시간 대기하는 것을 Backoff 라고 합니다.
- 일시적 장애 회복 시간 제공 : retry 하기 전에 term 을 두어서 회복시간 부여

retry-exception:

- 어떤 exception 발생시 재시도 할지

ignore-exceptions:

- 어떤 exception 발생시 재시도를 하지 않을지

```
retry:
  configs:
    # 기본 재시도 설정
    default:
      max-attempts: 3
      wait-duration: 500ms
      retry-exceptions:
        - feign.RetryableException
        - java.io.IOException
        - java.net.SocketTimeoutException
        - java.util.concurrent.TimeoutException
      ignore-exceptions:
        - feign.FeignException$BadRequest
        - feign.FeignException$Unauthorized
        - feign.FeignException$Forbidden
        - feign.FeignException$NotFound
```



# Case (8) - circuit breaker

다음은 **default** 에 대한 설정입니다.

sliding-window-type: COUNT\_BASED

- 슬라이딩 윈도우 타입 (COUNT\_BASED(호출횟수 기반), TIME\_BASED(시간 기반))

sliding-window-size: 100

- 슬라이딩 윈도우 크기. 최근 N개의 호출을 기준으로 실패율을 계산

minimum-number-of-calls: 10

- 최소 호출 횟수. 이 횟수 이상 호출되어야 circuit breaker 가 실패율을 계산

failure-rate-threshold: 50

- 실패율 임계값(%) : 이 비율 이상 실패시 Circuit OPEN

slow-call-rate-threshold: 80

- 느린 호출 기준 시간. 이 시간 이상 걸릴 경우 느린 호출로 간주

wait-duration-in-open-state: 60s

- OPEN 상태 대기 시간. Circuit 이 OPEN 된 후 HALF\_OPEN 으로 전환되기까지의 대기 시간

permitted-number-of-calls-in-half-open-state: 10

- HALF\_OPEN 상태에서 허용할 호출 수. 이 횟수 만큼 테스트 호출 후 CLOSED/OPEN 결정

automatic-transition-from-open-to-half-open-enable: true

- 자동 전환 활성화 : OPEN → HALF\_OPEN 자동 전환 여부

```
# Resilience4j 설정
resilience4j:
  circuitbreaker:
    configs:
      # 기본 설정 (일반적인 서비스)
      default:
        # 슬라이딩 윈도우 타입: COUNT_BASED(호출 횟수 기반) 또는 TIME_BASED(시간 기반)
        sliding-window-type: COUNT_BASED
        # 슬라이딩 윈도우 크기: 최근 N개의 호출을 기준으로 실패율 계산
        sliding-window-size: 100
        # 최소 호출 횟수: 이 횟수 이상 호출되어야 Circuit Breaker가 실패율을 계산
        minimum-number-of-calls: 10
        # 실패율 임계값(%): 이 비율 이상 실패 시 Circuit OPEN
        failure-rate-threshold: 50
        # 느린 호출 비율 임계값(%): 이 비율 이상이 느린 호출이면 Circuit OPEN
        slow-call-rate-threshold: 80
        # 느린 호출 기준 시간: 이 시간 이상 걸리면 느린 호출로 간주
        slow-call-duration-threshold: 5s
        # OPEN 상태 대기 시간: Circuit이 OPEN된 후 HALF_OPEN으로 전환되기까지 대기 시간
        wait-duration-in-open-state: 60s
        # HALF_OPEN 상태에서 허용할 호출 수: 이 횟수만큼 테스트 호출 후 CLOSED/OPEN 결정
        permitted-number-of-calls-in-half-open-state: 10
        # 자동 전환 활성화: OPEN → HALF_OPEN 자동 전환 여부
        automatic-transition-from-open-to-half-open-enabled: true

      # 중요 서비스용 (Timeline, Member 등 사용자 UX에 직접 영향)
      critical:
        sliding-window-type: COUNT_BASED
        sliding-window-size: 50      # 빠른 감지를 위해 작은 윈도우 사용
        minimum-number-of-calls: 5   # 5번만 호출되어도 패턴 감지
        failure-rate-threshold: 40    # 더 민감한 임계값 (40% 실패 시 OPEN)
        slow-call-rate-threshold: 70  # 느린 호출에 대해서도 민감하게 반응
        slow-call-duration-threshold: 3s # 3초 이상이면 느린 호출
        wait-duration-in-open-state: 30s # 빠른 복구 시도 (30초)
        permitted-number-of-calls-in-half-open-state: 5 # 적은 테스트 호출로 빠른 결정
        automatic-transition-from-open-to-half-open-enabled: true
```

# Case (8) - retry

**retry** : 현재 서비스에서 다른 마이크로서비스로 **feign** 을 통해 요청을 호출시 에러 발생시 재시도 횟수, 재시도 전 **timeout** 정도, 어떤 **Exception** 에 대해 재시도할지 등에 대한 재시도 설정

- 불필요한 재시도 방지 (e.g. 4xx 등에 대해서는 즉시 실패 처리)
- 의미있는 재시도 (e.g. 네트워크 장애, 타임아웃의 경우 재시도하도록 설정)
- Backoff 전략으로 부하 분산 (백오프 : 시스템이 실패 후 재시도할 때 즉시 재시도하지 않고 일정시간 대기하는 전략을 의미합니다.)

설정의 정도에 따라 **default**, **fast**, **conservative** 로 분류해두었고, 각 서비스별로 다음과 같이 지정했습니다.

fast : timelineService, memberService

default : contentService, activityService

conservative : imageService

다음은 **default** 에 대한 설정입니다.

max-attempts: 3

- 최대 3번까지 시도 (최초 1번 + 재시도 2번), 실패 시 총 3번의 호출 시도 후 최종 실패 처리

wait-duration: 500ms

- 각 재시도 사이의 대기 시간 (실패 후 재시도 하지않고 500ms 를 기다린 후 재시도)

- 장점
  - Backoff : 대상 서비스에 즉시 재시도하지 않고 일정시간 대기하는 것을 Backoff 라고 합니다.
  - 일시적 장애 회복 시간 제공 : retry 하기 전에 term 을 두어서 회복시간 부여

retry-exception:

- 어떤 exception 발생시 재시도 할지

ignore-exceptions:

- 어떤 exception 발생시 재시도를 하지 않을지

**Backoff** 에는 다음의 종류들이 있습니다.

**Fixed Backoff** (고정 백오프) : 매번 동일한 시간 대기 (e.g. 500ms → 500ms → 500ms)

**Exponential Backoff** (지수 백오프) : 재시도마다 대기 시간이 지수적으로 증가 (e.g. 100ms → 200ms → 400ms → 800ms)

**Linear Backoff** (선형 백오프) : 재시도마다 대기 시간이 선형적으로 증가 (e.g. 100ms → 200ms → 300ms → 400ms)

```
retry:
  configs:
    # 기본 재시도 설정
    default:
      max-attempts: 3
      wait-duration: 500ms
      retry-exceptions:
        - feign.RetryableException
        - java.io.IOException
        - java.net.SocketTimeoutException
        - java.util.concurrent.TimeoutException
      ignore-exceptions:
        - feign.FeignException$BadRequest
        - feign.FeignException$Unauthorized
        - feign.FeignException$Forbidden
        - feign.FeignException$NotFound
```

# Case (8) - rate limiter

**rate limiter** : 현재 서비스에서 다른 마이크로서비스로 **feign** 을 통해 요청을 호출하는 **outbound** 요청의 빈도(rate)를 제한하는 기능

- self-protection : 다른 서비스에 과부하를 주지 않도록 보호해줄수 있습니다.
- 연쇄 장애 방지 : 현재 서비스에서의 과도한 요청으로 인해 전체 시스템에 영향을 주지 않도록 합니다.
- 호출 주기 안정화 : 특정 서비스로의 급격한 트래픽 증가시 제한을 걸수 있습니다.
- 일정한 양의 최소/최대 호출 가능 rate 를 지정하면, 현재 시스템 내에서 해당되는 대상 서비스 역시 어느 정도의 트래픽을 허용치로 돌지 명확해집니다.

설정의 정도에 따라 **default, critical, conservative** 로 분류해두었고, 각 서비스별로 다음과 같이 지정했습니다.

critical : timeline-svc, member-svc

default : content-svc, activity-svc

conservative : image-svc

다음은 **default** 에 대한 설정입니다.

limit-refresh-period : 1s

- 제한 갱신 주기
- 1초로 지정했습니다.
- 1초마다 기록했던 rate 가 리셋됩니다.

limit-for-period : 100

- 갱신 주기 당 허용 호출 수
- limit-refresh-period 구간 내에서 ‘허용할 호출 수’ 를 의미합니다.
- 1초에 100번 호출 가능하다는 것을 의미합니다.

timeout-duration : 0s

- 허용량 초과시 대기시간
- 0s 는 대기하지 않고 즉시 실패하는 것을 의미합니다.

```
ratelimiter:
  configs:
    # 기본 Rate Limiter 설정

    default:
      # 제한 갱신 주기: 1초마다 제한이 리셋됨
      limit-refresh-period: 1s

      # 갱신 주기당 허용 호출 수: 1초에 100번 호출 가능
      limit-for-period: 100

      # 허용량 초과 시 대기 시간: 대기하지 않고 즉시 실패
      timeout-duration: 0s


    # 중요 서비스용 (더 많은 요청 허용)

    critical:
      limit-refresh-period: 1s

      limit-for-period: 200 # 1초에 200번 호출 가능 (부하가 높을 수 있음)

      timeout-duration: 100ms # 100ms까지 대기 후 실패


    # 보수적 Rate Limiter (외부 서비스 보호)

    conservative:
      limit-refresh-period: 1s

      limit-for-period: 50 # 1초에 50번으로 제한 (외부 서비스 부담 감소)

      timeout-duration: 0s


  instances:
    timelineService:
      base-config: critical # Timeline은 트래픽이 많으므로 여유있게

    memberService:
      base-config: critical

    contentService:
      base-config: default

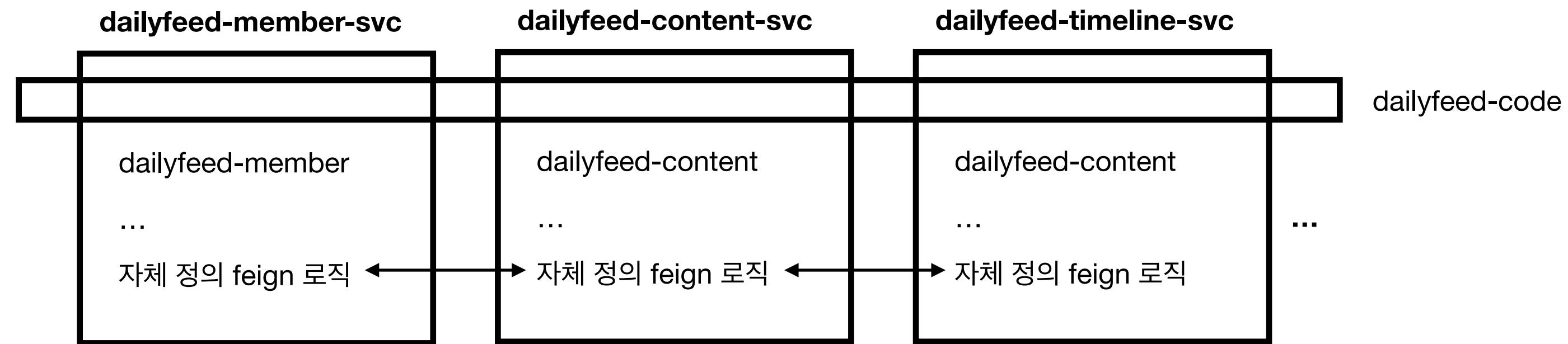
    activityService:
      base-config: default

    imageService:
      base-config: conservative # Image 서비스는 부담이 크므로 제한
```

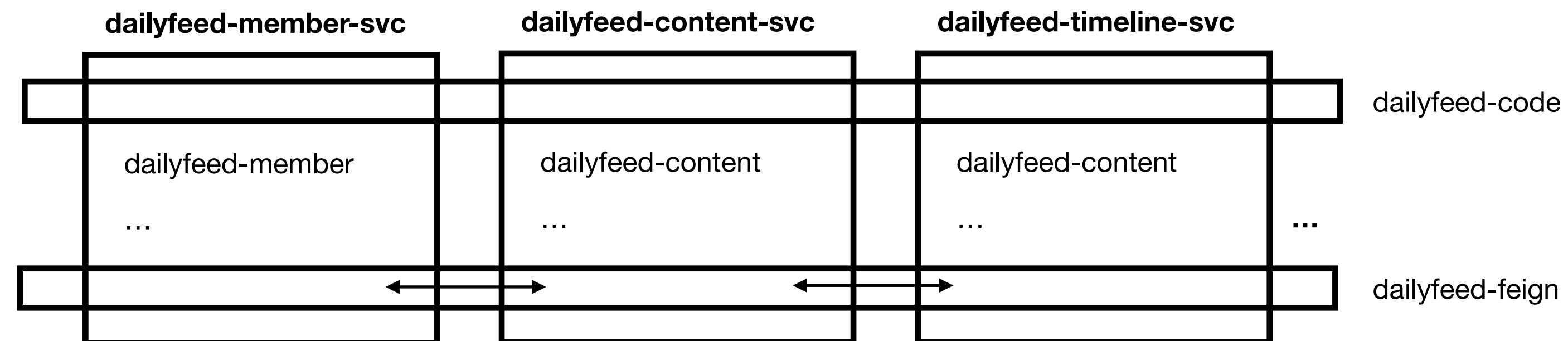


# Case (8) 서브모듈(dailyfeed-feign)으로 분리

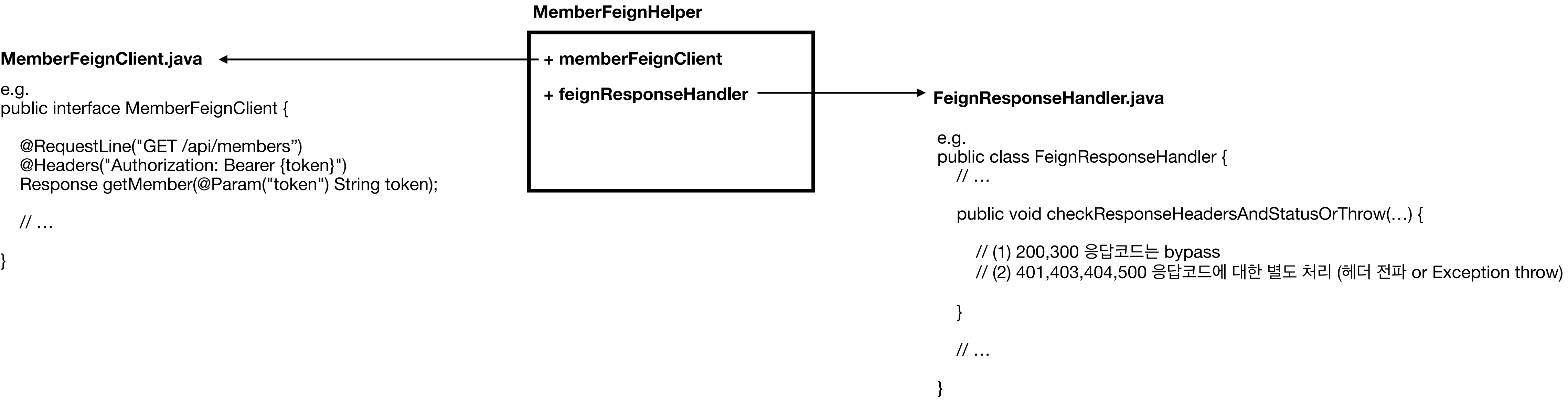
e.g. feign 기능/엔드포인트를 서비스 별로 개별 정의해서 사용시



e.g. feign 기능/엔드포인트의 코드를 모듈로 추출 및 정의 후 통신 인터페이스 역할을 수행하도록 정의



# Case (8) FeignHelper, FeignClient, FeignResponseHandler



**Case (9) JWT**

**: Key Refresh**

**: JWT ID, Black List**

**: ArgumentResolver**



# Case (9) JWT - 요약

## 인증 방식 : **ArgumentResolver**

AOP 를 이용해 JWT 를 분해하고 검사하거나, member-svc 로 유효한 사용자인지를 조회할 수도 있겠지만

ArgumentResolver 방식을 통해 인증을 수행하는 방식을 선택했습니다.

Argument 나 파라미터를 검증한다는 목적에 부합하고, AOP 대신 ArgumentResovler 기반의 인증방식을 선택했는데

지나친 AOP 의존은 공통로직의 비대화와 결합도 증가를 불러온다고 생각하기에 ArgumentResolver를 선택했습니다.

## 서비스간 통신

서비스간 통신에는 JWT 를 주고받으면서 인증된 사용자인지 검증합니다.

AccessToken 이 만료되었을 경우에는

401 Unauthorized, {X-Token-Refress-Needed: true} 응답 헤더를 전파해서 POST /api/token/refresh 를 하도록 유도합니다.

RefreshToken 이 만료되었을 경우에는

401 Unauthorized, {X-Relogin-Required: true} 응답 헤더를 전파해서 login 을 새로 수행하도록 유도합니다.

## **MemberProfileSummaryArgumentResolver**

- **@AuthenticatedMemberProfileSummary**



MemberFeignHelper, MemberFeignClient 를 사용해서 GET /api/members/profile 요청 수행

성공시 MemberProfileDto.MemberProfileSummary 를 반환

dailyfeed-member-svc 의 인증로직을 가지고 있지 않은 일반 서비스에서 사용

e.g.

- dailyfeed-content-svc, dailyfeed-timeline-svc, dailyfeed-search-svc, dailyfeed-image-svc, dailyfeed-activity-svc

## **AuthenticatedMemberInternalArgumentResolver**

- **@InternalAuthenticatedMember**

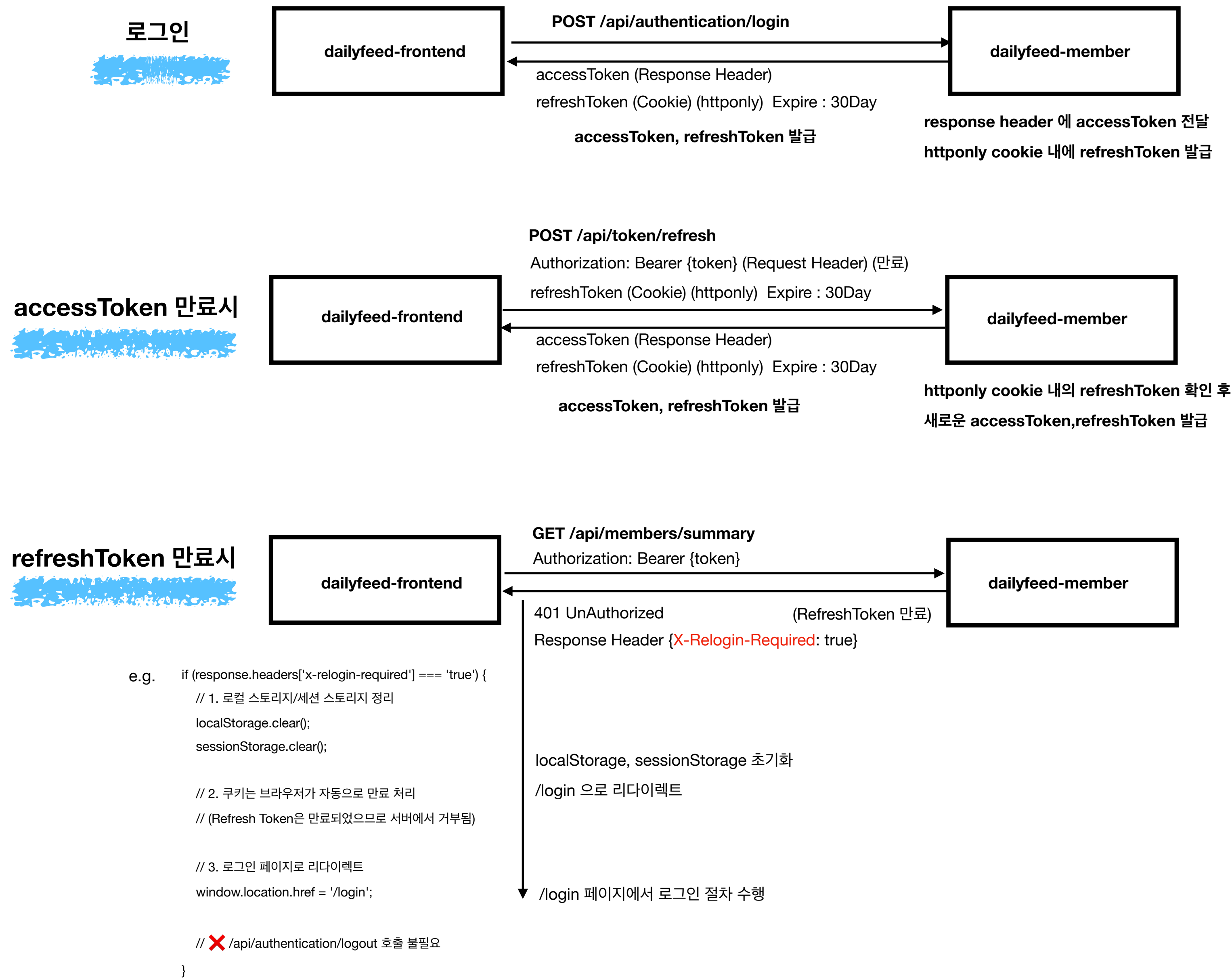


GET /api/members/profile 내에서 내부 인증 조회를 수행

성공시 MemberDto.Member 를 반환

인증로직을 포함하고 있는 dailyfeed-member-svc 에서 사용

## Case (9) JWT - 요약





# Case (9) JWT - Key Refresh, Refresh Token, Blacklist

## Access Token, Refresh Token

- Access Token 은 사용자가 인증되었음을 의미하는 JWT 입니다. 짧은 만료기한을 가지며, Access Token 이 만료되었을 경우 401 Unauthorized 를 응답하며, 응답 헤더로 {'X-Token-Refresh-Needed': 'true'} 를 return 합니다.
- Refresh Token 은 새로운 accessToken, refreshToken 을 부여받을때 사용하는 토큰입니다. 위와 같이 {'X-Token-Refresh-Needed': 'true'} 를 응답받을때 refreshToken 으로 서버에 POST /api/token/refresh 요청을 합니다.

e.g.

- 로그인: 로그인 시 응답헤더에는 {'Authorization': 'Bearer {토큰값}'} 을 담아서 응답, cookie 에는 Refresh Token 을 응답 (만료: 30Day)
- AccessToken 만료시 : 클라이언트에서는 만료된 AccessToken 에 대해 401 Unauthorized, {'X-Token-Refresh-Needed': 'true'} 를 응답받습니다.
  - 이후 frontend 는 member-svc 백엔드에 POST /api/token/refresh 를 호출해서 새로운 accessToken, refreshToken 을 받은 후 하던 작업을 재요청합니다.
- Server 측 동작 :
  - 이 때 member-svc (서버) 에서는 기존 refresh token 철회(revoke)하고, 새로운 refresh token 을 발급, 그리고 새로운 access token 역시 발급해서 access token(응답헤더), refresh token(쿠키) 을 응답하는 과정을 거칩니다.

## Blacklist

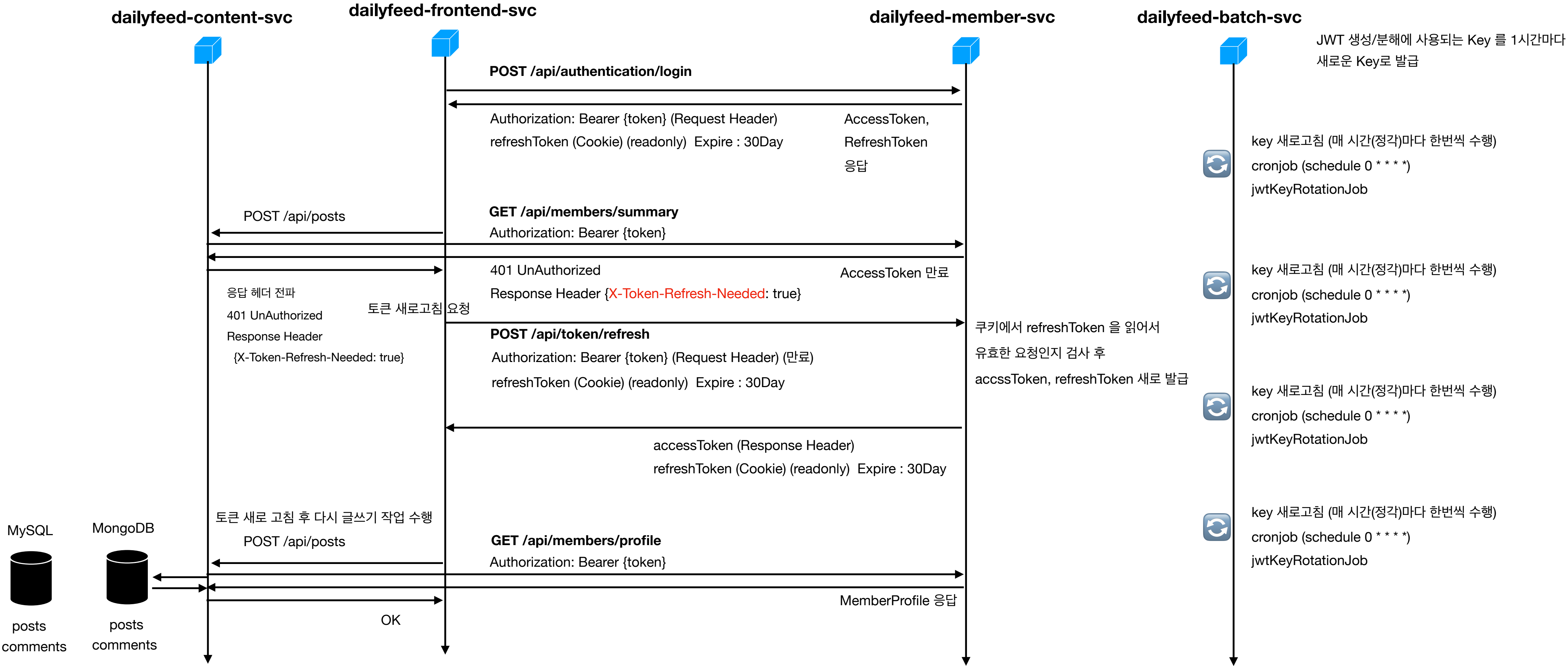
- 로그아웃을 했을 때 로그아웃된 해당 토큰을 Blacklist 에 추가해서, 중복된 토큰을 사용하지 않도록 보장합니다.
- 만료된 토큰,로그아웃된 토큰을 악용하는 케이스에 대응할수 있습니다.
- 만약 Blacklist 에 해당되는 토큰으로 API 요청이 발생할 경우 서버에서는 401 Unauthorized 를 응답하며 응답헤더로 {'X-Relogin-Required': 'true'} 를 응답하게 됩니다. (feign 호출의 경우 Header/Status 전파)
- 클라이언트는 401 Unauthorized, 응답헤더 {'X-Relogin-Required': 'true'} 를 응답받으면 sessionStorage, localStorage 를 무효화하고, /login 페이지로 사용자를 리다이렉트 시킵니다.

## JWT Key, Key Refresh

- JWT 생성 용도의 비공개 Key 는 dailyfeed-batch 에 의해 1시간에 한번씩 새로운 Key 로 바뀌며 해당 key 는 'jwt\_keys' 라는 테이블에 저장됩니다. (cronjob 기반)
- JWT 생성 용도의 비공개 Key 의 kid 는 JWT 내의 'kid' 항목에 담겨서 보내집니다.
- 서버로 JWT를 담아 요청 시에 해당 JWT 내의 'kid' 의 값이 오래된 값일 경우 (이미 새로운 Key 가 생성되어 있는 경우) 서버에서는 응답 헤더에 {'X-Token-Refresh-Needed': 'true'} 를 담아서 정상 응답을 합니다.
- 클라이언트는 이 응답을 받으면 하던 작업을 모두 완료한 후 별도로 POST /api/token/refresh 로 요청을 보내 accessToken, refreshToken 을 새롭게 부여받습니다.
- 서버 애플리케이션 초기 배포시 JWT Key 값이 INSERT 되어야 하는데, 이 경우 helm 설치 쉘 스크립트에서 python 파일을 통해 초기 키값을 생성하도록 정의해두었습니다.

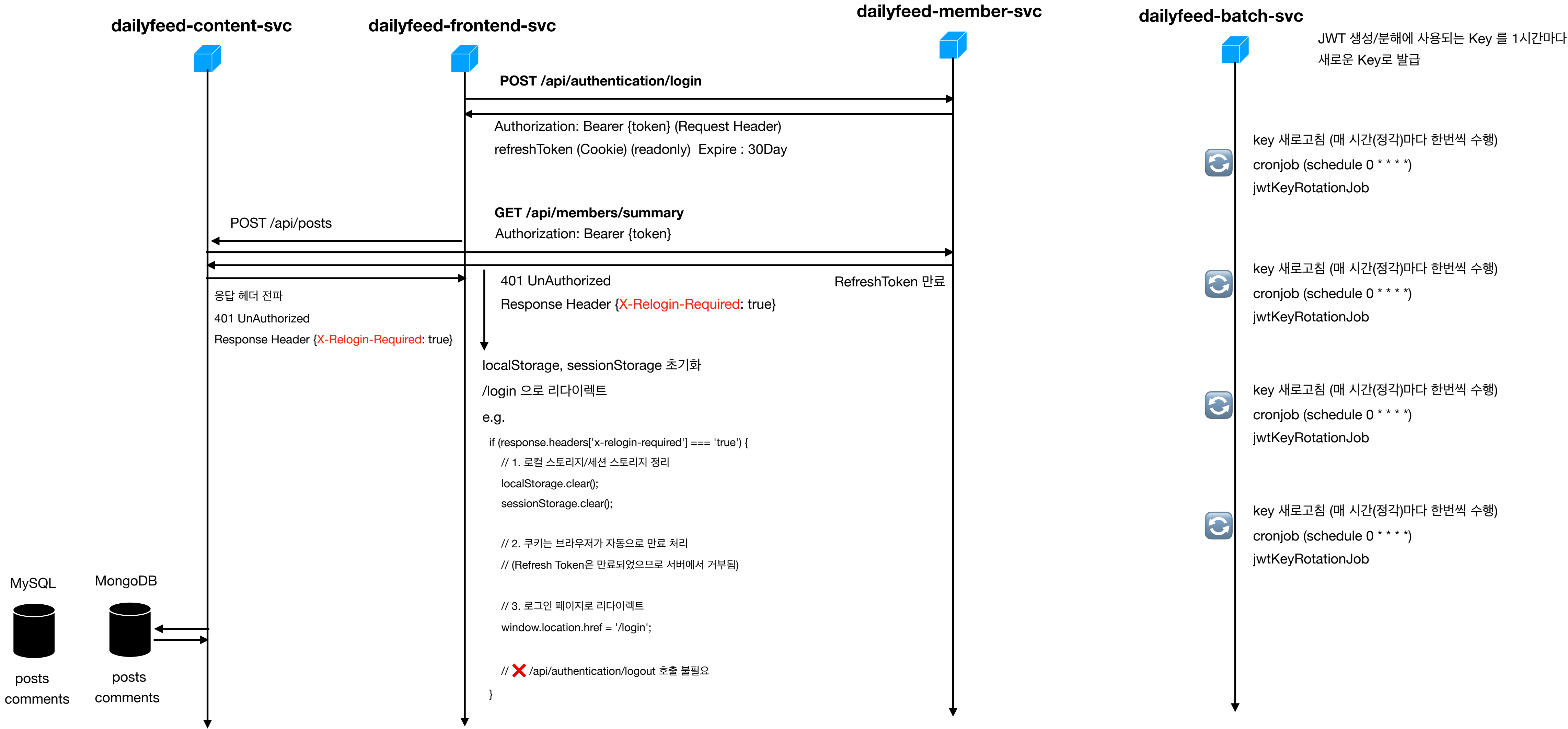
# Case (9) JWT - Access Token 만료(Expired) 시 동작

AccessToken 이 만료될 경우  
Refresh Token 을 이용해서 POST /api/token/refresh 를 수행 합니다.  
member-svc 에서는 새로운 Access Token 은 응답헤더로, 새로운 Refresh Token 은 cookie 에 세팅



# Case (9) JWT - Refresh Token 만료(Expired) 시 동작

서버에서는 RefreshToken 이 만료될 경우 응답 헤더 {X-Relogin-Required: true} 와 함께 401 Unauthorized 를 응답합니다.  
클라이언트에서는 token 무효화 + /login 페이지로 이동합니다.





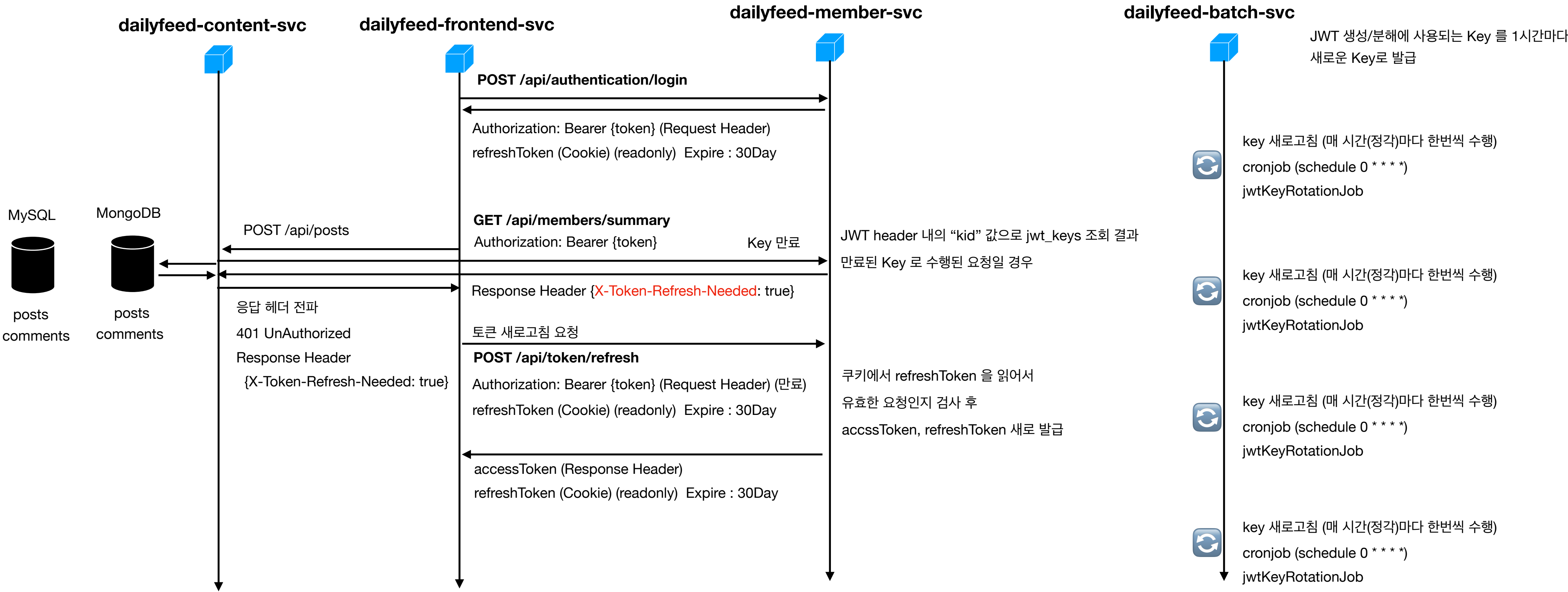
# Case (9) JWT - 오래된 “kid” 로 만들어진 JWT 로 요청이 왔을 경우

## 만료된 Key 로 만든 JWT 로 요청시

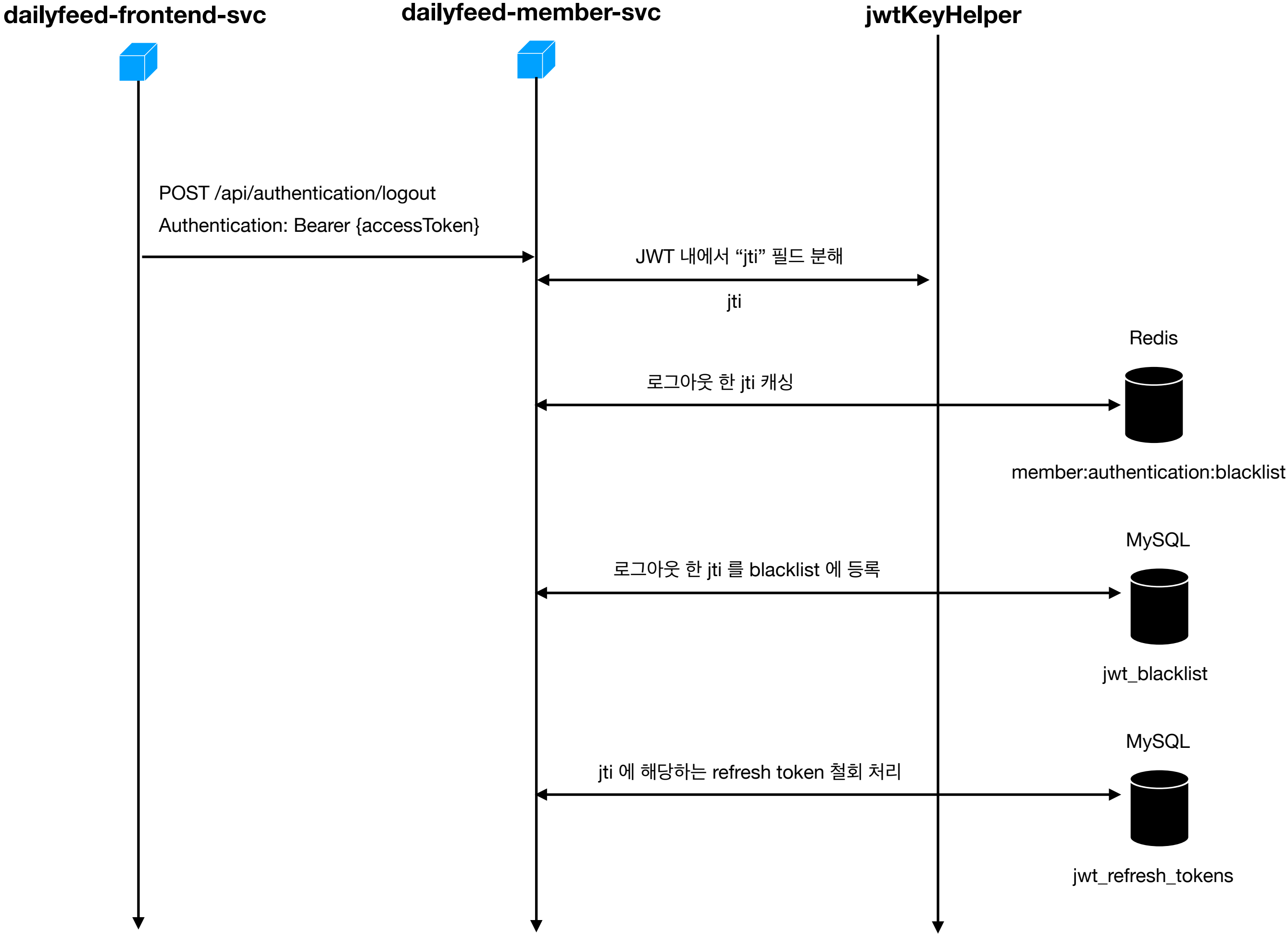
JWT Header 내의 “kid” 값으로 “jwt\_keys” 테이블 조회 후, 찾은 Key 가 만료된 Key(is\_active=false)일 경우

응답 헤더로 {X-Token-Refresh-Needed: true} 를 응답하고 클라이언트에게 정상응답을 하며, 클라이언트 역시 하던 작업을 정상 수행합니다.

해당 작업 수행 후 별도로 클라이언트에서 서버로 **POST /api/token/refresh** 요청을 보내 새로운 accessToken, refreshToken 을 부여받습니다.

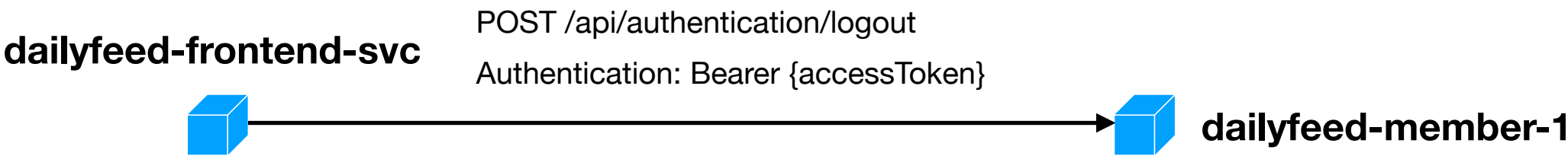


# Case (9) JWT - 로그아웃

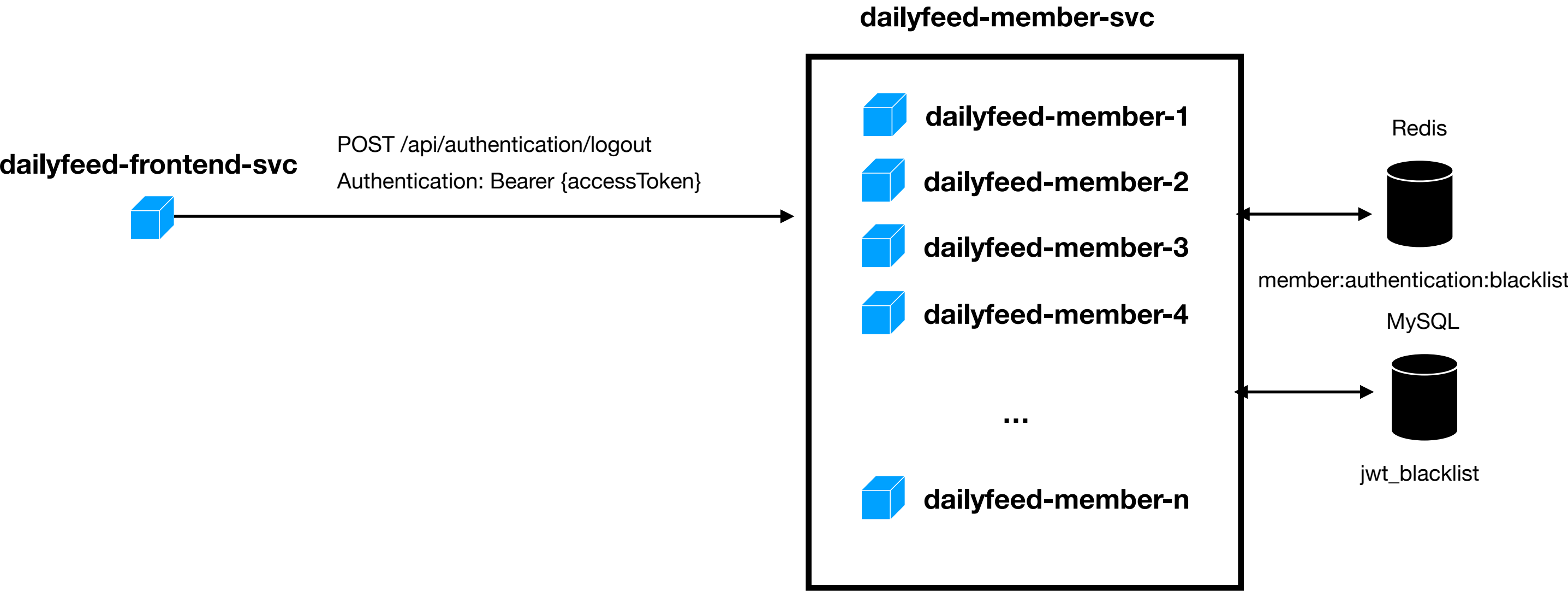


# Case (9) JWT - Blacklist

단일 인스턴스로 구성 시



n 개의 인스턴스로 구성 시

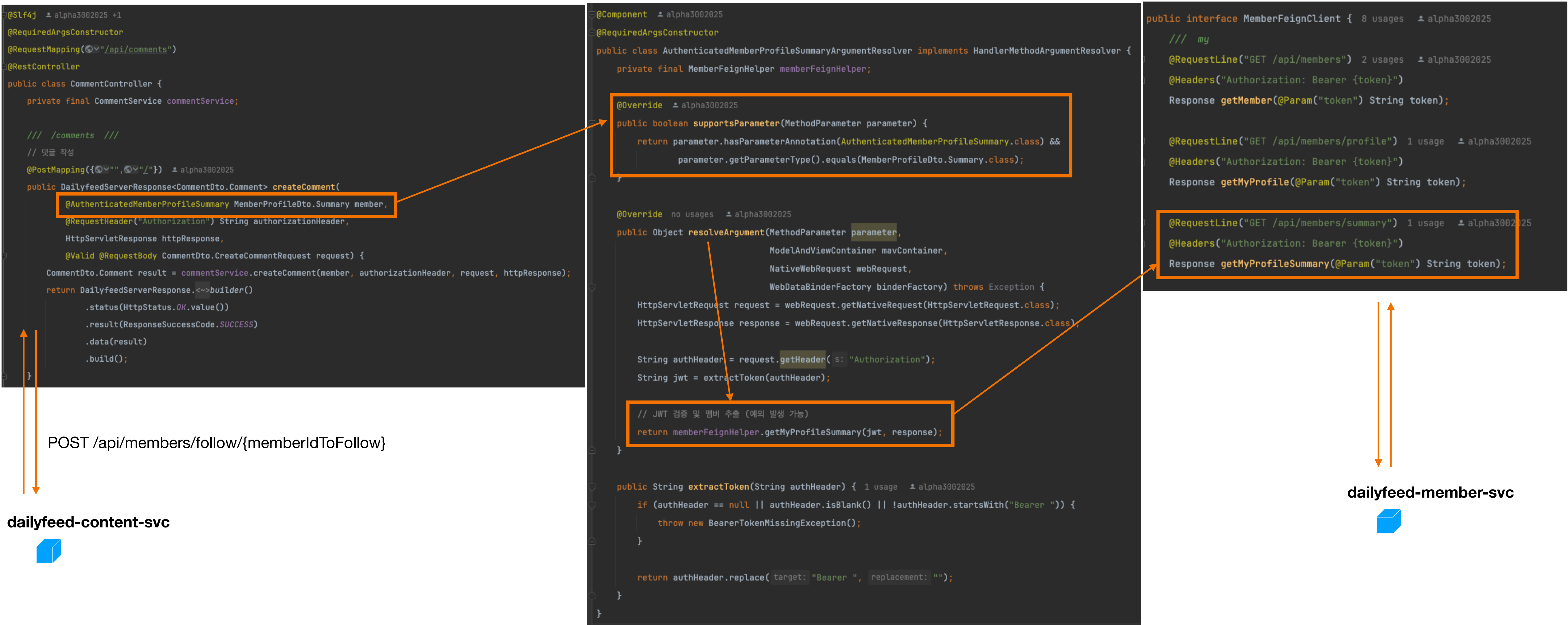




# Case (9) JWT - MemberProfileArgumentResolver

## MemberProfileArgumentResolver

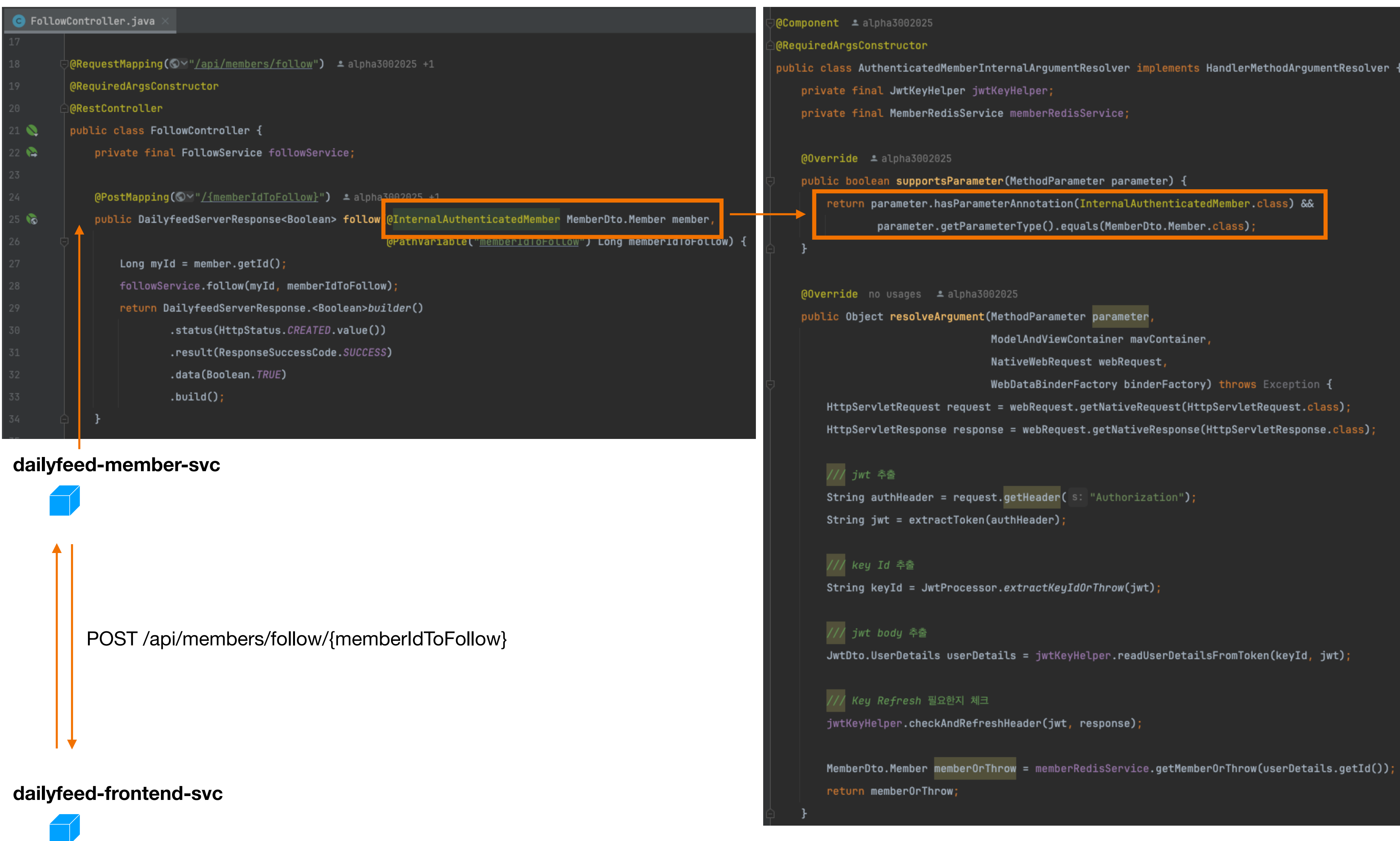
- content-svc, timeline-svc, search-svc, image-svc, activity-svc 등에서는 dailyfeed-member-svc 로 GET /api/members/profile 을 통해 요청한 사용자가 유효한 사용자인지를 조회합니다.
- 이 로직은 거의 모든 ‘@RequestMapping’ 에 사용되는데, 해당 request 가 처리 되기 전에 ArgumentResolver 에서 다음과 같이 memberFeign 을 통해 member-svc 에 JWT가 유효한 사용자인지 조회하게 됩니다.



## Case (9) JWT - AuthenticatedMemberInternalArgumentResolver

## AuthenticatedMemberInternalArgumentResolver

dailyfeed-member-svc 로 요청이 왔을때 (e.g. 팔로우 요청)는 AuthenticationMemberInternalArgumentResolver 를 사용합니다. dailyfeed-member-svc 내에서 내부적으로 인증된 사용자인지를 검증하기 위한 ArgumentResolver 입니다.





# Case (9) JWT - ArgumentResolver 선택 이유

## ArgumentResolver 선택 이유

처음에는 AOP 또는 interceptor 를 고려했습니다. 하지만 다음의 단점으로 인해 조금 더 유연하고 요청에 맞는 파라미터를 어노테이션 기반으로 골라서 선택을 할수 있다는 장점으로 인해 ArgumentResolver 를 선택했습니다.

## AOP

- 처음에는 AOP 사용시 편하지만 나중으로 갈수록 비대해지는 common 코드처럼 변질
- AOP 내에서 수행해야 하는 동작들이 많아져서 코드의 결합도가 올라가며, AOP 코드를 수정하는데에 대한 두려움 증가

## Interceptor

- API 의 request path 에 따라 유연하게 선택할수도 있겠지만, api versioning 등을 수행할 경우 interceptor 역시 수정해야 한다는 점이 신경쓰였습니다.
- JWT 의 검증 등의 수행이 interceptor 로 요청을 검증하는 성격과는 많이 이질감이 있기 때문에 interceptor 는 선택 대상에서 제외했습니다.

# Case (10) Follow 정책, 댓글/답글 정책

# Case (10) Follow 정책, 댓글/답글 정책

## Follow 정책

- 한 사람이 1000명 이상을 follow 하지 못하도록 막아두었습니다.
- 나를 팔로우 하는 팔로워가 100만명이 되는 것은 상관이 없지만, 팔로잉의 경우 1000명 이상을 팔로우하는 것은 금지해두었습니다. 즉 999명 까지만 팔로우가 가능합니다.
- 999명 이상을 팔로우하는 것은 악용되는 사용자인거나 bot 의 가능성이 있을수 있고, 피드를 조회할 때 1000명 이상 멤버에 대한 피드를 조회해야 하는 케이스가 생기기 때문입니다.

## 댓글/답글 허용 레벨

- 답글은 3레벨 까지만 작성이 가능합니다. 답글 삭제시 중첩된 삭제가 발생할 가능성이 높기 때문에 가급적 3레벨까지 허용하는 것이 맞다고 판단했습니다.
- reddit, twitter 와 같은 정보 공유와 정보의 소스가 중요하면서 개인의 의견을 중요하게 생각하는 플랫폼의 경우 댓글/답글 레벨 역시 무한에 가깝게 되어 있지만 dailyfeed 의 경우 커뮤니티의 성격이 더 강하기에 3레벨 내에서 소통이 가능하다고 생각했기 때문입니다.

## 댓글/답글 삭제시 정책

- 댓글 삭제 시 자식 답글들은 모두 삭제 됩니다.
- 답글 삭제 시 자식 답글들 역시 모두 삭제 됩니다.
- reddit, twitter 와 같은 정보공유/정보의 소스, 개인의 의견을 중요하게 생각하는 플랫폼의 경우 ‘삭제된 글입니다’로 표시 후 자식 댓글/답글을 표현하지만, dailyfeed 의 경우에는 커뮤니티의 성격이 더 강하고 사생활 측면이 더 중요하다고 느껴 댓글/답글 삭제시 자식 댓글/답글 삭제를 하도록 했습니다.

## 댓글/답글 API 는 별도로

- 댓글 작성 API 와 답글 작성 API 를 POST /api/comments 에 두는 것도 생각할수 있지만, 경험상 답글의 경우 댓글과 정책이 조금씩 달라지는 케이스가 있기에 댓글 작성 API, 답글 작성 API 는 별도로 분리했습니다.
- API 하나에서 두 가지 이상의 기능으로 분기를 하는 레거시 코드를 접한 경험이 있기에 두 기능을 별도의 API로 분리해서 독립적으로 수정 또는 관리되도록 구성했습니다.



# Case (11) 주요 코딩 컨벤션

: no common → 서브모듈로 공유, 지나친 data 모듈 구성 자제, 수직계열화 방지

: dailyfed-code (인터페이스 역할의 Enum, Dto, Exception 등 정의)

: 'OOOMapper'

: get—OrThrow

: boolean (x) → Predicate enum 정의

# Case (11) 주요 코딩컨벤션 : no common → 서브모듈로 공유, data 모듈 구성 자제, 수직계열화 방지

## no common → 서브모듈로 공유

- 현재 프로젝트에서는 common 모듈을 사용하지 않습니다.
- common(x) → dailyfeed-code, dailyfeed-redis-support, dailyfeed-kafka-support, dailyfeed-pagination-support, dailyfeed-deadletter-support
- common 모듈 대신 dailyfeed-code, dailyfeed-redis-support, dailyfeed-kafka-support, dailyfeed-pagination-support, dailyfeed-deadletter-support 와 같은 구체 모듈로 분리했습니다.
- 성능 측면에서 모든 코드를 core 또는 common 등에 모아두는 것은 자제
- 실제로 common 또는 core 모듈을 사용하던 시스템을 경험했을때, 신규 서비스 구현시 common 에서 소모하는 스레드풀,메모리로 인해 신규서비스 자체가 고혈압에 걸린 뚱뚱한 서비스가 나오는 것을 지켜봤었습니다.
- “common”, “core” 모듈을 사용하지 않는 것으로 인해 **해야 할 작업이 정말 많아졌지만**, 처음 결정한 대로 계속해서 어떤 것을 모듈화하고 개별 서비스 간 “common”,”core” 에 대한 지나친 종속성으로 인한 불편함은 방지했습니다.
- 또 다른 관점에서는 “지나친 공통화에 대한 욕심”이 결국은 “어리석은 똑똑함” 처럼 발전할 때가 있다고 자주 느꼈기 때문에 이번 프로젝트에서는 “common”, “core” 모듈구현은 자제했습니다.

## data 모듈 구성 자제

- 쿼리(JPA, SQL, Querydsl) 들을 특정 모듈에 모아두지 않았습니다.
- 쿼리를 공통화하면, 처음에는 정말 편하지만 나중에 갈수록 이 쿼리가 다른 서비스에서 쓰는 것인지 등에 대한 확신이 점점 없어지고, 레거시 쿼리 들이 점점 부비트랩 처럼 변한다고 생각했기에 data 모듈을 구성하는 것은 자제했습니다.
- 예를 들어 dailyfeed-content-svc, dailyfeed-timeline-svc 에서 Post, Comment 라는 도메인은 각 서비스별로 서로 다른 쿼리(JPA, SQL, Querydsl)로 사용합니다.
- 즉, dailyfeed-content-svc 입장에서는 Post,Comment 의 작업은 독립성을 가지고 쿼리를 구성하고, dailyfeed-timeline-svc 입장에서 Post,Comment 의 작업을 독립성을 가지고 쿼리를 구성하도록 했습니다.
- 이렇게 하면 dailyfeed-content-svc 를 수정하기 위해 쿼리(JPA, SQL, Querydsl)를 수정했을 때 dailyfeed-timeline-svc 에 영향을 주지 않고 독립성을 가질수 있습니다.
- 구현을 편하게 하고 빠르게 하려면, 쿼리(JPA, SQL, Querydsl)를 한 곳에 모아뒀겠지만, 서비스별로 독립성을 부여하기 위해 이렇게 각각 별도로 구체 쿼리들을 개별 서비스들이 보유하도록 구성했습니다.

## 수직계열화 방지

- 코드 구성시 가급적 수직계열화 되는 현상을 방지하기 위해 노력했습니다.
- 어떤 라이브러리, 어떤 버전의 라이브러리를 사용한다는 것을 규격화 하기 위한 지나치게 중첩화되고 지나치게 규격화된 서브모듈 사용방식은 지양했습니다.
- 라이브러리의 선택은 각 서비스별로 자율적으로 선택해서 할수 있도록 하는게 낫다고 판단했기 때문입니다.
- 예를 들어 JWT 분해 로직을 여러 모듈에 공유하고 싶다고 해서 dailyfeed-code 에 넣어두진 않았습니다. JWT 분해 로직은 member-service 만 필요하게끔 서비스를 구성했고, 외부로 노출시키지 않았기에 불필요하게 공통 로직을 만들지는 않았습니다. 만약 module 간 공유가 필요했다면 dailyfeed-jwt-support 라는 모듈로 분리했을 것입니다.

# Case (11) 주요 코딩컨벤션 : dailyfeed-code (인터페이스 역할의 Enum,Dto,Exception 정의)

## dailyfeed-code

- 서비스간 API 를 통해 통신을 할때 사용되는 Dto, Exception, Enum 은 dailyfeed-code 라는 서브모듈에 정의했습니다.
- dailyfeed-code 는 common 모듈처럼 동작하는 모듈이 아닙니다. 공통코드는 아닙니다.
- dailyfeed-code 에는 특정 함수나 비즈니스로직을 포함하지 않고, 상수, Enum, 예외코드, Exception, 데이터 표현 객체(Dto) 만을 담는 기능이 되도록 구성했습니다.
- 즉 dailyfeed-code 는 어떤 값이나 상수, 예외코드, Exception, 데이터 표현 객체(Dto)만을 담고 있으며, 서비스간 통신 시 또는 모듈간 기능 혼합시 “인터페이스” 역할을 하게 됩니다.





# Case (11) 주요 코딩컨벤션 : ‘OOMapper’

## ‘OOMapper’ 컴포넌트

- 객체를 생성하거나 변환하는 로직은 ‘OOMapper’ 에서 수행하도록 정의했으며, ‘OOMapper’ 는 Spring 이 관리하는 Component 가 되도록 정의했습니다.
  - 개별 비즈니스 로직 내에서 객체를 하드코딩으로 생성할 경우, 테스트가 어려워지는 단점도 있고 객체 생성 로직이 이곳 저곳에 하드코딩으로 존재하게 되어 일괄적인 수정이 불가능해지는 케이스가 많습니다.
  - 또한 하드코딩된 객체 생성 로직은 레거시 코드 수정시 해당 기능에 대한 확신을 떨어뜨려서 수정하기 어려운 코드로 만들게 됩니다.
  - 이런 이유로 객체 생성 코드는 가급적 ‘OOMapper’ 컴포넌트가 담당하도록 했고, 프로젝트 내에서 98% 정도의 코드에서 ‘OOMapper’ 를 강제해서 사용했습니다.
- (나머지 2%는 기한상의 이유로 하드코딩된 코드가 조금 있습니다.)
- 처음에는 ‘OOMapper’ 를 구현하면서 컴포넌트 명,함수 명을 짓는 것부터 손이 가는 부분들이 자잘하게 많았지만, 코드를 수정하는 과정에서는 관리가 더 쉬워지는 것을 경험해서 장점이 명확하다고 느꼈습니다.

## dailyfeed-member / MemberProfileMapper

```
@Mapper(componentModel = "spring", unmappedTargetPolicy = ReportingPolicy.IGNORE) 13 usages  1 implementation  👤 alpha3002025
public interface MemberProfileMapper {

    MemberProfileMapper INSTANCE = Mappers.getMapper(MemberProfileMapper.class); no usages


    default MemberProfileDto.MemberProfile fromEntity(MemberProfile memberProfile, Long followersCount, Long followingsCount) {

        return MemberProfileDto.MemberProfile.builder()

            .id(memberProfile.getId())

            .memberId(memberProfile.getMember().getId())

            .memberName(memberProfile.getMemberName())

            .handle(memberProfile.getHandle())

            .displayName(memberProfile.getDisplayName())

            .bio(memberProfile.getBio())

            .location(memberProfile.getLocation())

            .websiteUrl(memberProfile.getWebsiteUrl())

            .birthDate(memberProfile.getBirthDate())

            .gender(memberProfile.getGender())

            .languageCode(memberProfile.getLanguageCode())

            .countryCode(memberProfile.getCountryCode())

            .verificationStatus(memberProfile.getVerificationStatus())

            .privacyLevel((memberProfile.getPrivacyLevel() == null) ? null : memberProfile.getPrivacyLevel())

            .profileCompletionScore(memberProfile.getProfileCompletionScore())

            .isActive(memberProfile.getIsActive())

            .avatarUrl(memberProfile.getAvatarUrl())

            .coverUrl(memberProfile.getCoverUrl())

            .createdAt(memberProfile.getCreatedAt())

            .updatedAt(memberProfile.getUpdatedAt())

            .followersCount(followersCount)

            .followingsCount(followingsCount)

            .build();

    }

}
```

## dailyfeed-content / CommentMapper

```
@Component 2 usages  👤 alpha3002025 +1
public class CommentMapper {

    public CommentDto.Comment fromCommentNonRecursive(Comment comment, MemberProfileDto.Summary author){ 3 usages

        return CommentDto.Comment.builder()

            .id(comment.getId())

            .content(comment.getContent())

            .authorId(comment.getAuthorId())

            .authorName(author != null ? author.getDisplayName() : MessageProperties.K0.DELETED_USER)

            .authorHandle(author != null ? author.getMemberHandle() : MessageProperties.K0.DELETED_HANDLE)

            .authorAvatarUrl(author != null ? author.getAvatarUrl() : MessageProperties.K0.NO_AVATAR_URL)

            .postId(comment.getPost().getId())

            .parentId(comment.getParent() != null ? comment.getParent().getId() : null)

            .depth(comment.getDepth())

            .createdAt(comment.getCreatedAt())

            .updatedAt(comment.getUpdatedAt())

            .build();

    }

}
```



# Case (11) 주요 코딩컨벤션 : ‘get—OrThrow()’

## ‘get—OrThrow()’

- 예외를 던지는 함수에는 가급적 ‘-OrThrow()’ 접미사를 붙였습니다.
- 트랜잭션 처리나 인증로직 구현 시에 특정 함수가 예외를 던지는지 아닌지를 일일이 찾아보게 되는데, 이런 이유로 가급적 함수가 예외를 던질수 있다는 의미를 명확히 하기 위해 ‘-OrThrow()’ 접미사를 사용했습니다.
- 현재 프로젝트에서는 80% 이상의 로직에서 ‘-OrThrow()’ 메서드를 사용하고 있습니다.

## e.g. dailyfeed-feign / FeignResponseHandler

```
public void checkResponseHeadersAndStatusOrThrow(Response feignResponse, HttpServletResponse httpResponse){  
    final int status = feignResponse.status();  
    if (status >= 200 && status < 300) {  
        return;  
    }  
  
    if (status >= 400 && status < 500) {  
        // HTTP 상태 코드에 따른 적절한 예외 처리  
        if (status == 401) {  
            log.error("Unauthorized request to member service - invalid or expired token");  
            propagateTokenRefreshHeader(feignResponse, httpResponse);  
            throw new MemberUnauthorizedException();  
        } else if (status == 403) {  
            log.error("Forbidden request to member service - insufficient permissions");  
            throw new MemberForbiddenException();  
        } else if (status == 404) {  
            log.error("Member not found in member service");  
            throw new MemberNotFoundException();  
        } else if (status >= 500) {  
            log.error("Member service internal error - status: {}", status);  
            throw new MemberApiConnectionErrorException();  
        } else {  
            log.error("Unexpected member service error - status: {}", status);  
            throw new MemberApiConnectionErrorException();  
        }  
    }  
}
```

## e.g. dailyfeed-member / AuthenticationService

```
@Transactional(readOnly = true) 1 usage  a1pha3002025  
public Member getMemberOrThrow(AuthenticationDto.LoginRequest loginRequest) {  
    return memberRepository  
        .findFirstByEmailFetchJoin(loginRequest.getEmail())  
        .orElseThrow(() -> new MemberNotFoundException());  
}  
  
public void checkIfPasswordMatchesOrThrow(String requestPassword, String encryptedPassword) {  
    if (!passwordEncoder.matches(requestPassword, encryptedPassword)) {  
        throw new MemberPasswordInvalidException();  
    }  
}
```



# Case (11) 주요 코딩컨벤션 : ‘boolean (x) → Predicate Enum’

## ‘boolean (x) → Predicate Enum’ 사용

- 주로 인증로직에서 어떤 로직이 true/false 를 return 되더라도 Expired 됐다는건지 Not Expired 라는 것인지 의미가 모호해지는 케이스가 많습니다.
- 현재 프로젝트에서는 70% 이상의 코드에서 boolean return 대신 Enum 을 return 하도록 구성했습니다. (간단한 boolean return 이더라도 의미가 명확하다면 굳이 Enum 을 사용하지는 않았습니다.)

### dailyfeed-member / AuthenticationService

```
public DailyfeedServerResponse<MemberDto.Member> signup(AuthenticationDto.SignupRequest signupRequest) {  
    if (MemberExistsPredicate.EXISTS.equals(checkIfMemberAlreadyExists(signupRequest))) {  
        throw new MemberAlreadyExistsException();  
    }  
  
    Member newMember = authenticationMapper.newMember(signupRequest, passwordEncoder, roles: "MEMBER");  
    Member saved = memberRepository.save(newMember);  
  
    return DailyfeedServerResponse.<~>builder()  
        .status(HttpStatus.CREATED.value())  
        .result(ResponseSuccessCode.SUCCESS)  
        .data(authenticationMapper.fromMemberEntityToMemberDto(saved))  
        .build();  
}
```

### dailyfeed-member / AuthenticationService

```
@Transactional(readOnly = true) 1 usage  alpha3002025  
public MemberExistsPredicate checkIfMemberAlreadyExists(AuthenticationDto.SignupRequest signupRequest) {  
    if (memberRepository.findFirstByEmailFetchJoin(signupRequest.getEmail()).isPresent()) {  
        return MemberExistsPredicate.EXISTS;  
    }  
  
    return MemberExistsPredicate.NOT_EXISTS;  
}
```

### dailyfeed-member / JwtAuthenticationFilter

```
// 블랙리스트 확인  
if (BlackListedPredicate.BLACKLISTED.equals(tokenService.isTokenBlacklisted(jti))) {  
    log.debug("Token is blacklisted: JTI={}", jti);  
    addReLoginRequiredAtResponseHeader(response);  
    return;  
}
```

### dailyfeed-member / TokenService

```
public class TokenService {  
    public BlackListedPredicate isTokenBlacklisted(String jti) { 1 usage  alpha3002025  
        try{  
            // 1. Redis 에서 먼저 체크  
            Boolean isKeyExists = stringRedisTemplate.hasKey(blacklistedJtiRedisKey(jti));  
            if(Boolean.TRUE.equals(isKeyExists)){  
                return BlackListedPredicate.BLACKLISTED;  
            }  
  
            // 2. Redis 에 없을 경우 DB 확인  
            Optional<TokenBlacklist> blacklist = tokenBlacklistRepository.findByJti(jti);  
            if (blacklist.isEmpty()) {
```

## Case (13) 어려웠던 점들

# 어려웠던 점들

## 비용 문제로 kubernetes cluster 안에 infra 를 직접 설치해야 했던 점

- : 개인 프로젝트이기에 비용을 최소화 하기 위해 kubernetes cluster 내에 redis,mysql,kafka,mongodb 를 설치했는데, 이 infra 들에 대한 작업을 하는 데에 시간이 정말 많이 걸렸습니다.
- : 돈이 충분히 있다면 redis,mysql,mongodb 정도는 클라우드 서비스를 사용했을 것 같습니다.
- : 위의 infra 리소스들로 인해 애플리케이션 hpa 설정시 각각의 requests, limits 를 점진적으로 늘리면서 infra 리소스가 죽는 현상이 없는지를 일일이 테스트하고 kubectl top 명령을 통해 마지막으로 안정적이었던 사양을 메모하는 거쳤는데 이 과정이 시간이 꽤 걸렸습니다.

## dev 용도로 infra 별도 구성작업

- : dev 프로필 용도의 rds 를 매일 9시에 생성되었다가 18시에 삭제되도록 boto 를 이용해서 rds, cloudwatch, lambda, security group 을 생성하고 삭제하는 것을 자동화했던 경험
- : Claude 웹 에서 자료를 조사한 내용을 바탕으로 수월하게 적용했지만, 처음에 어떤식으로 접근할지 구상하는 과정이 쉽지 않았습니다.

## helm chart/tmeplate 구성작업

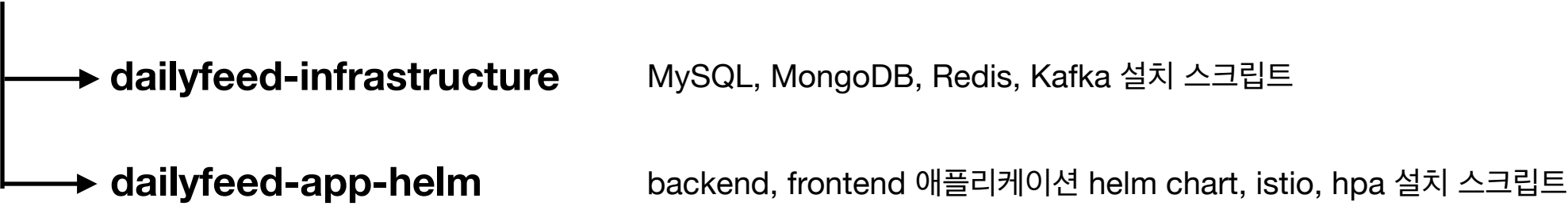
- : helm chart/template 구성 작업 시 어떤 필드명을 고쳐야한다...는 생각이 들때 그에 따르는 다른 코드들을 고치고 패키징하고 하는 과정에서 피로도를 꽤 느꼈습니다.
- : helm 에서 사용할 values-local.yaml, values-dev.yaml, values.yaml 을 base chart 에서는 어떤 것을 사용하고 각각의 애플리케이션에서는 어떤 것을 사용하고, 필드 경로의 중첩구조는 어떻게 잡고 하는 과정에서 고민을 정말 많이했었던 것 같습니다.
- : 막상 helm 구성하고나면 명령어 하나로 해당 서비스를 install/uninstall/upgrade 가 가능하다는 점은 정말 편리하다고 느꼈습니다.



## Case (14) dailyfeed-installer 구조 설명

# Case (14) dailyfeed-installer 구조

## dailyfeed-installer



## dailyfeed-installer

e,g,

local profile : source local-install-infra-and-app.sh main

dev profile : source dev-install-infra-and-app.sh main

